

ОПЕРАЦИОННЫЕ СИСТЕМЫ **LINUX** ДЛЯ **ХАКЕРОВ** И НЕ ТОЛЬКО...



НАСТОЛЬНАЯ КНИГА ПЕНТЕСТЕРА

Оглавление

ПРЕДИСЛОВИЕ	5
ГЛАВА 1. АРХИТЕКТУРА.....	7
Историческая справка.....	7
Дистрибутивы.....	8
Архитектура.....	9
Многозадачность	9
Сигналы	11
Системные вызовы (syscall).....	12
Сокеты	14
Дисковая и файловые подсистемы	15
Таблица разделов.....	17
Файловая подсистема.....	18
СТРУКТУРА КАТАЛОГОВ	20
Специальные файловые системы.....	23
Основные функции файловой системы	23
Аудит	24
Управление логическими томами (LVM).....	28
УПРАВЛЕНИЕ ПАКЕТАМИ	29
УПРАВЛЕНИЕ ПОЛЬЗОВАТЕЛЯМИ И ПРИВИЛЕГИЯМИ	31
Модель AAA.....	32
Атрибуты и права файлов.....	33
Расширенные атрибуты	34
Пользователи и группы	35
Управление пользователями и группами	37
BASH СКРИПТЫ	40
Особенности работы	41
Принципы автоматизации	42
Практика написания скриптов.....	42
Параметры.....	44
Цикл for и while	45
Цикл while.....	47
Функции	49
ИТОГИ.....	51
ГЛАВА 2. СЕТИ И СЕТЕВЫЕ СЕРВИСЫ	52
СЕТЕВАЯ АРХИТЕКТУРА И МАРШРУТИЗАЦИЯ	52
Протокол L2 ARP	54
Проблемы ARP протокола	55
Технология VLAN.....	55
Протокол L3 IP.....	55
Маршрутизация L3.....	57
Протокол OSPF.....	57
Протоколы TCP и UDP.....	57
Порты TCP и UDP.....	59
Протоколы L7.....	59
Telnet.....	60
SSH.....	60
DNS	61
DHCP.....	62

СЕТЕВЫЕ УТИЛИТЫ	63
<i>Wireshark</i>	63
<i>Tcpdump</i>	65
<i>Применение фильтров</i>	66
<i>Ngrep</i>	67
ФИЛЬТРАЦИЯ ТРАФИКА	68
<i>Iptables</i>	68
<i>Критерий правила</i>	70
<i>Действие правила</i>	71
<i>Firewalld</i>	71
<i>Nftables</i>	73
СЕРВИСЫ НА БАЗЕ LINUX (WEB/MAIL/SQL SERVERS)	73
<i>Web-серверы</i>	73
<i>Apache</i>	74
<i>Почтовые сервисы в Linux</i>	78
<i>Postfix</i>	79
<i>Dovecot</i>	81
<i>DKIM, DMARC, SPF</i>	82
<i>Базы данных в Linux</i>	83
<i>PostgreSQL</i>	87
ГЛАВА 3. БЕЗОПАСНОСТЬ В ОС LINUX	89
СБОР ЛОГОВ	89
<i>Приложения для ведения логов в Linux</i>	90
<i>Утилиты просмотра и взаимодействия с файлами логов</i>	93
ОБЩИЕ НАСТРОЙКИ БЕЗОПАСНОСТИ ОС LINUX	94
<i>SELinux</i>	98
<i>AppArmor</i>	99
<i>Polkit или PolicyKit</i>	103
<i>TCP Wrappers</i>	103
АУДИТ БЕЗОПАСНОСТИ ОС LINUX	104
VPN	106
<i>Виртуальные интерфейсы в Linux</i>	109
ИТОГИ	112
ГЛАВА 4. СИСТЕМЫ ХРАНЕНИЯ И АНАЛИЗА ЛОГОВ	113
ЗАЧЕМ НУЖНЫ СИСТЕМЫ ХРАНЕНИЯ ЛОГОВ	113
<i>Файл конфигурации</i>	113
<i>Настройка сервера для сбора логов</i>	116
<i>Возможные категории для логов</i>	117
<i>Возможные категории для логов (severity)</i>	117
СРАВНЕНИЕ ПОПУЛЯРНЫХ СИСТЕМ ХРАНЕНИЯ ЛОГОВ	118
<i>Rsyslog</i>	118
<i>Logstash</i>	119
<i>Graylog</i>	122
РУЧНОЙ АНАЛИЗ ЛОГОВ	122
<i>Поиск логов</i>	123
<i>О правилах записи строк в логах</i>	124
<i>Поиск в лог-файлах</i>	126
<i>Приведение к читабельному виду</i>	128
JOURNAL	130
<i>Установка</i>	130

Особенности сбора данных.....	131
Journal-upload.....	132
GRAYLOG.....	133
Установка.....	133
Установка MongoDB для Graylog.....	136
Установка Graylog.....	137
Настройка Graylog.....	137
Настройка сбора логов.....	140
ИТОГИ.....	144
ГЛАВА 5. РАЗВЕДКА И ЗАКРЕПЛЕНИЕ.....	145
СБОР ИНФОРМАЦИИ О СИСТЕМЕ.....	145
СБОР ИНФОРМАЦИИ О ПОЛЬЗОВАТЕЛЯХ, ГРУППАХ, ПАРОЛЯХ.....	147
БАЗОВЫЕ ТЕХНИКИ ПОВЫШЕНИЯ ПРИВИЛЕГИЙ.....	149
Sudo misconfiguration.....	149
Kernel exploits.....	151
Buffer overflow.....	151
SUID (SET USER ID).....	153
Что такое разрешение SUID?.....	154
Как SUID помогает в повышении привилегий?.....	155
SUID и команда find.....	155
Cron.....	156
Missing libraries.....	159
ПРОДВИНУТЫЕ ТЕХНИКИ ПОВЫШЕНИЯ ПРИВИЛЕГИЙ.....	162
Directory/Path traversal.....	162
Type Confusion.....	165
Race Condition.....	166
Integer overflow.....	167
ДОПОЛНИТЕЛЬНЫЕ ТЕХНИКИ ПОВЫШЕНИЯ ПРИВИЛЕГИЙ.....	168
Apt-get.....	168
nmap.....	170
Dirty Cow.....	170
Process injection.....	172
Proc/[pid]/mem.....	173
VDSO hijacking.....	174
ИТОГИ.....	174
ЗАКЛЮЧЕНИЕ.....	175

Предисловие

Летом 2023 года (в то самое время, когда ныне покойный товарищ Пригожин организовывал свой поход на Москву, а доллар стоил меньше 80 рублей), я осознал, что, призывая практически в каждом видео (да я немножко видеоблогингом балуюсь, [ссылка кликабельна](#)) делать акцент на матчасти и базе, не сообщая подписчикам в чём же конкретно, по моему мнению, заключается эта самая база.

Особенности архитектуры операционных систем Windows и Linux, сетевые сервисы, логи и системы хранения, нюансы разведки и закрепления, повышение привилегий, работа с дампами памяти, атаки, фильтрация трафика, маршрутизация, конфигурирование Firewall и базовое администрирование с использованием Powershell (будь он проклят).

И это лишь то небольшое, что пришло мне в голову в тот момент (и осталось на память в виде корявого текста на обратной стороне чека из местной пятёрочки). Вернувшись из отпуска, данная задумка выкристаллизовалась в намерение сделать полноценный видеокурс для начинающих (и уже весьма опытных) пентестеров. Само-собой с акцентом на «белых» хакеров, как наиболее востребованных сегодня специалистов на рынке ИБ и ИТ в целом.

Примерно полгода ушло на то, чтобы собрать наброски из теоретических материалов, набрать команду высококлассных экспертов из разных направлений кибербезопасности (кодеры, БДшники, пентестеры веба, сетевики ~~и прочие прилебатели~~), проработать с ними практическую часть курса, оформить это дело в виде классных сценариев и...осознать, насколько же это гигантский объём.

Даже по самым скромным прикидкам на запись и монтаж всех этих роликов ушло бы не меньше 10 лет. А это, на минуточку, примерно столько, сколько я в принципе веду свой канал на ютубе. И, разумеется, ни временных, ни уж тем более финансовых ресурсов (чтобы прожить то время, пока буду записывать курс, не отвлекаясь на основную работу) у меня пока нет и в обозримом будущем не появится.

Пару месяцев я дико прокрастинировал по этому поводу, оплакивая вложенные силы и деньги (консультационные услуги высококлассных экспертов в ИТ нынче стоят дороже эскортниц), попутно уходя с головой в другие, менее вдохновляющие проекты, продавая фрагменты наиболее удачных сценариев на кворке и фриланс.ру (так что не удивляйтесь, если вдруг где-то услышите\уже слышали нечто похожее).

В то время по вечерам я обычно придавался своему любимому хобби - чтению технической литературы. Собственно, именно эта активность в конечном итоге и привела меня к гениальной (во всяком случае очень хочется в это верить) идее, просто изложить наработанный материал в виде серии книг. Да, пусть это не совсем то, что мне изначально хотелось бы видеть. Комьюнити книголюбов значительно меньше аудитории на ютубе.

И соответственно донести свои мысли получится лишь до тех, кто за годы деграционных процессов в системы образования каким-то чудом ещё не забыл, как выглядят буквы. Однако, положив руку на сердце, само качество людей, не утративших понимания пользы от чтения значительно выше. Так что, как минимум, можно наверняка быть уверенным в том, что в руки «случайных прохожих» данная книга точно не попадёт.

И если ты сейчас читаешь данные строки, то, по сути, ужеходишь в 2% избранных. Остальные 98%, если когда-нибудь и смогут ознакомиться с этой инфой, то потратят на её самостоятельное освоение и «набивание шишек» минимум лет 10-15 (и то в случае, если наставник попадётся толковый, а суровая жизнь не внесёт свои коррективы).

В данной книге я подробнейшим образом изложил свой собственный опыт и наработки экспертов из индустрии в части эксплуатации операционных систем с точки зрения тестирования на проникновения. И если бы я мог одним щелчком пальцев вернуться в прошлое и прочесть её, будучи студентом шараги или возрастным гуманитарием пожелавшим что-то изменить в жизни и пойти по пути кибербезопасности, я бы незамедлительно это сделал.

Поэтому я искренне, белой завистью вам завидую. Ведь ваша дорожная карта специалиста, после прочтения, будет содержать значительно меньше ухабов и ответвлений с тропинками «в никуда». По сути, это гладкий асфальт, по которому вам просто нужно проехать (возможно не один раз). Так что, если готовы, устраивайтесь по удобней, и будем уже заканчивать эту порядком затянувшуюся прелюдию. Погнали душнить!

Глава 1. Архитектура

В современном мире практически нигде не встретишь IT-ландшафт в котором представлена только одна операционная система (например, Windows). Именно поэтому знание Linux сегодня очень востребовано (а в ситуации с импортозамещением за этот скилл ещё и платят вдвойне).

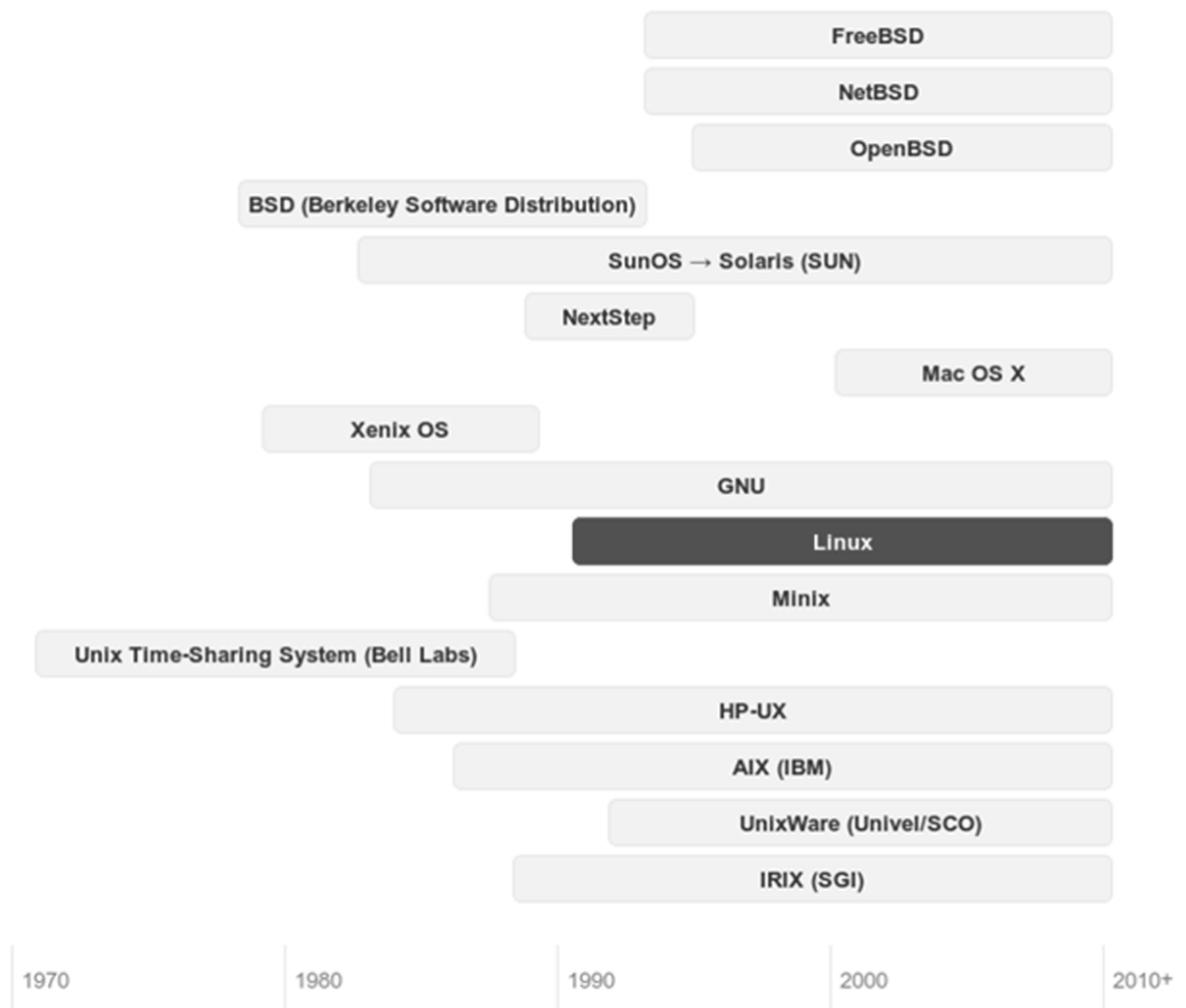
Вам, как будущим (и настоящим) специалистам по тестированию на проникновение, очень пригодится понимание основ Linux, т.к. частенько различные сервисы (в том числе на сетевом периметре организации) работают именно под управлением Linux (почтовые сервера, веб-приложения и т.д.).

В этой главе мы с вами изучим:

- общие принципы работы с операционной системой Linux;
- основные утилиты встречающиеся в Linux;
- принципы построения логирования и защиты системы;
- наиболее распространенные задачи, которые решает операционная система Linux;
- подходы к автоматизации повседневных задач с помощью простых Bash-скриптов.

Историческая справка

Давайте немного отмотаем время назад и разберёмся, откуда и как появился этот злосчастный *Linux*. В самом начале, а именно в 1969 году, в США появилась **операционная система UNIX** представляющая собой основу систем промышленного класса. Распространялась данная ОС приблизительно за 40.000\$ (офигеть, правда?). Получить её исходные коды или внести какие-то изменения в ядро, разумеется, никак не представлялось возможным. Так что именно цена и невозможность изменения под свои нужды и послужило в итоге толчком к появлению в 1983 году **GNU** проекта, который будет распространяться свободно и окажется совместим с *UNIX*, но при этом не будет ей полностью идентичен. Единственная загвоздка, препятствующая выпуску такой вот чудо системы заключалась в отсутствии ядра. Его лишь в 1991 году разработал небезызвестный фин Линус Торвальдс (чем и прославился). Своё детище, основанное на прототипе *Minix*, Линус назвал *Linux* и выложил в свободный доступ (какой молодец). Первые версии ОС *Linux* были разработаны на базе процессора *Intel x86*.



Дистрибутивы

Linux – это всего лишь ядро операционной системы на базе которого в свою очередь построено много различных дистрибутивов. Каждый из них решает свои задачи, поэтому невозможно сказать, какой дистрибутив «лучше» всех (хотя иной раз за хороший донат очень хочется). Перечислять здесь все дистрибутивы — задача бессмысленная, они уже перечислены [тут](#). Однако «базовые» дистрибутивы всё-таки имеет смысл обозначить. *Debian*, *Arch*, *CentOS* (земля пухом) и *Red Hat* («коммерческая» версия *CentOS*), *openSUSE*, *Gentoo*, *SlackWare*, *Android*...

У каждой из сборок есть свои достоинства и недостатки, однако может быть и такое, что вы ~~конечный-привереда~~ работаете над уникальной задачей и ни один из указанных дистрибутивов прям из коробки вам однозначно не подойдёт. Что же в таком случае спрашивается делать? Ответ прост — соберите своё собственное ядро и конечный дистрибутив. Тут вы сами в праве выбирать командный интерпретатор, модули ядра, графическую оболочку (или её отсутствие). Вообще кастомные сборки *Linux* могут встречаться в самых необычных местах (например, в некоторых IoT устройствах и в металлургической промышленности на контроллерах). Попробуйте и вы на досуге собрать что-то своё (если конечно руки прямые и из нужного места растут).

Архитектура

В *Linux*, как и в любой другой операционной системе, может быть 2 варианта архитектуры: 32 (раньше эта архитектура ещё называлась *x86* из-за процессоров *Intel*, серия которых заканчивалась на «86») и 64 бита. Что же такое **архитектура**? Это структура, определяющая порядок обработки информации и взаимодействие железа с софтом. Отличия 32 и 64-битных версий одной и той же внешне никак не выражены, однако под капотом у них всё же много отличий.

Первое (и самое важное отличие) — **способность использовать оперативную память**. В 32-битной версии любой ОС (неважно, *Linux* или *Windows*) можно использовать до 4-х гигабайт оперативной памяти. Именно это ограничение подтолкнуло 32-битные ОС и процессоры в небытие (формально, они ещё существуют в целях обратной совместимости, но по факту любой 64-битный процессор или ОС сможет выполнить любые функции 32-разрядной версии)

Второе отличие присутствует уже на стороне железа: 32-битные процессоры попросту **не смогут работать** с 64-битными приложениями или системами, тогда как 64-битные процессоры таких ограничений не имеют.

Многозадачность

Сегодня эра многопользовательских и многозадачных операционных систем. И *Linux* (кто б мог подумать), как раз относится к таковым. Далее мы подробнее разберёмся в понятиях **многопользовательский** и **многозадачный**. Многопользовательская система предполагает одновременную работу нескольких пользователей в одной ОС. То есть практически независимо друг от друга.

Условно один сервер используется программами (а под некоторые из них как раз создается отдельный пользователь, например, под веб-сервер) и администратором, под которого тоже нужен отдельный пользователь.

Например, вспомните турникеты на вход в офис — чтобы пройти, нужно приложить свою магнитную карточку сотрудника. Такая система тоже работает на *Linux*. Если бы ОС могла обрабатывать всего одну задачу, то одновременно приложить карточку двум людям было бы невозможно. Многозадачность предполагает одновременную работу многих процессов.

Предлагаю подробнее остановиться **на процессах**. Понятие одновременной работы процессов немного условное. Ядро ОС может очень быстро переключать процессорное время (читай мощность) между ними, настолько быстро, что мы даже не успеваем заметить, что они работают не одновременно.

Это происходит постоянно, так как ядер всегда меньше, чем процессов. На заре компьютерной эры было 1 ядро = 1 процесс, но в современных системах ядро мега-быстро переключается между выполнением разных задач.

Главная парадигма *Linux* гласит: **всё есть файл**. И процессы в *Linux* это тоже файлы. У каждого процесса в *Linux* есть обязательный набор атрибутов. Вот некоторые из них:

- **Процесс-родитель** (кроме процесса *init*, конечно же).
- **Идентификатор процесса (PID)** (у *init* PID всегда 1). Всегда положительное целое число. Обычно, выдается процессам последовательно (в общем случае — процесс с PID 1000 запустится позже процесса с PID 500).
- **Идентификатор процесса-родителя (PPID)** (у *init* PPID всегда 0. Это единственный процесс без родителя).

Исходя из этих атрибутов, можно сделать вполне себе логичный вывод: процессы в *Linux* могут порождать другие процессы. И у каждого процесса должен быть свой родитель. А что случится, если случайно (или намеренно) «убить» родителя?

В такой ситуации, возможно, всего 2 варианта развития событий: либо все дочерние процессы завершатся, либо им будет назначен новый родитель. В крайнем случае, это будет процесс *init*. Давайте попробуем посмотреть на процессы, которые сейчас запущены в системе *Linux*. Это можно сделать утилитами *top*, *ps* или *htop*.

```
kali@kali: ~
Applications 158.79.131/multit... x Burp Suite Community E... x Burp Suite
File Actions Edit View Help

0[ 0.0%] Tasks: 88, 312 thr; 1 running
1[ 0.0%] Load average: 0.07 0.05 0.01
2[ 0.7%] Uptime: 1 day, 15:48:05
3[ 1.3%]
Mem[|||||] 1.12G/1.93G
Swp[|||||] 380M/2.00G

PID PPID USER S Command
---
32101 24709 kali S qterminal
32102 24709 kali S qterminal
32108 24709 kali S qterminal
32109 32101 kali S /bin/bash
32128 32109 root S sudo -i
32129 32128 root S -bash
45012 32129 root R htop
32112 24709 kali S qterminal
32113 24709 kali S qterminal
32114 24709 kali S qterminal
32115 24709 kali S qterminal
32116 24709 kali S qterminal
32117 24709 kali S qterminal
32118 24709 kali S qterminal
32119 24709 kali S qterminal
32120 24709 kali S qterminal
32121 24709 kali S qterminal

F1 Help F2 Setup F3 Search F4 Filter F5 List F6 SortBy F7 Nice - F8 Nice + F9 Kill F10
```

В зависимости от различных обстоятельств состояние процесса во время работы может меняться. Вот основные состояния процессов в системе *Linux*:

- **Running (R)** — процесс выполняется или готов к работе. На скриншоте выше это утилита *htop* (собственно, она же и работает).
- **Sleep (S)** — процесс находится в состоянии прерываемого ожидания, то есть ожидает какого-то события, сигнала или освобождения нужного ресурса.
- **Zombie (Z)** — процесс не работает, но если он был остановлен, то возможно, в системе осталась выполняемая им задача.

Ядро системы делит процессы в состоянии ожидания на два типа:

- **Прерываемые процессы**, состояние ожидания которых может быть прервано сигналом (это любые процессы от пользователя).
Например, пользователь запустил какой-то сервис, но не использует его (Терминал тот же самый).
- **Непрерываемые**, состояние ожидания которых может прерваться только аппаратным способом (речь идет про системные процессы, которые запускает ядро. Пользователь не может им послать сигнал на включение).
Например, нельзя послать сигнал на включение USB или зажечь лампочку. USB-коннектор включится только тогда, когда вы вставите флешку в разъем, поступит ток, запустится процесс обработки файловой системы флешки.

Чтобы взаимодействовать с прерываемым процессом, используют сигналы. В большинстве разновидностей Linux встроена утилита **kill** для работы с процессами. Она позволяет отправлять сигналы определенным процессам, на которые укажет пользователь. Команда `sudo kill <номер сигнала> <pid>` используется для взаимодействия с процессами, например, для того чтобы убить процесс необходимо выполнить `sudo kill -9 <pid>`. Список доступных сигналов можно посмотреть командой `sudo kill -l`.

Давайте теперь разберемся с тем, что такое сигнал.

Сигналы

Сигнал в *Linux* — это программное прерывание, посылаемые процессу в случае возникновения какого-либо события (например, ошибки). Или, говоря простым языком, способ управления процессами. Большинство сигналов в ОС **асинхронны**, то есть необязательно привязаны к какому-либо событию.

Давайте попробуем разобраться в сигналах на примере «убийства» (т.е. сигнала завершения) процесса. Воспользуемся для этого утилитой *kill*.

```

root@kali:~# kill -l
 1) SIGHUP      2) SIGINT      3) SIGQUIT     4) SIGILL      5) SIGTRAP
 6) SIGABRT     7) SIGBUS     8) SIGFPE      9) SIGKILL     10) SIGUSR1
11) SIGSEGV    12) SIGUSR2    13) SIGPIPE    14) SIGALRM     15) SIGTERM
16) SIGSTKFLT  17) SIGCHLD   18) SIGCONT    19) SIGSTOP     20) SIGTSTP
21) SIGTTIN    22) SIGTTOU   23) SIGURG     24) SIGXCPU     25) SIGXFSZ
26) SIGVTALRM  27) SIGPROF   28) SIGWINCH   29) SIGIO        30) SIGPWR
31) SIGSYS     34) SIGRTMIN  35) SIGRTMIN+1 36) SIGRTMIN+2  37) SIGRTMIN+3
38) SIGRTMIN+4 39) SIGRTMIN+5 40) SIGRTMIN+6 41) SIGRTMIN+7  42) SIGRTMIN+8
43) SIGRTMIN+9 44) SIGRTMIN+10 45) SIGRTMIN+11 46) SIGRTMIN+12 47) SIGRTMIN+13
48) SIGRTMIN+14 49) SIGRTMIN+15 50) SIGRTMAX-14 51) SIGRTMAX-13 52) SIGRTMAX-12
53) SIGRTMAX-11 54) SIGRTMAX-10 55) SIGRTMAX-9  56) SIGRTMAX-8  57) SIGRTMAX-7
58) SIGRTMAX-6 59) SIGRTMAX-5 60) SIGRTMAX-4 61) SIGRTMAX-3  62) SIGRTMAX-2
63) SIGRTMAX-1 64) SIGRTMAX

```

В рамках данного материала подробно все сигналы описывать смысла не вижу, однако немного подробнее остановлюсь на базовых:

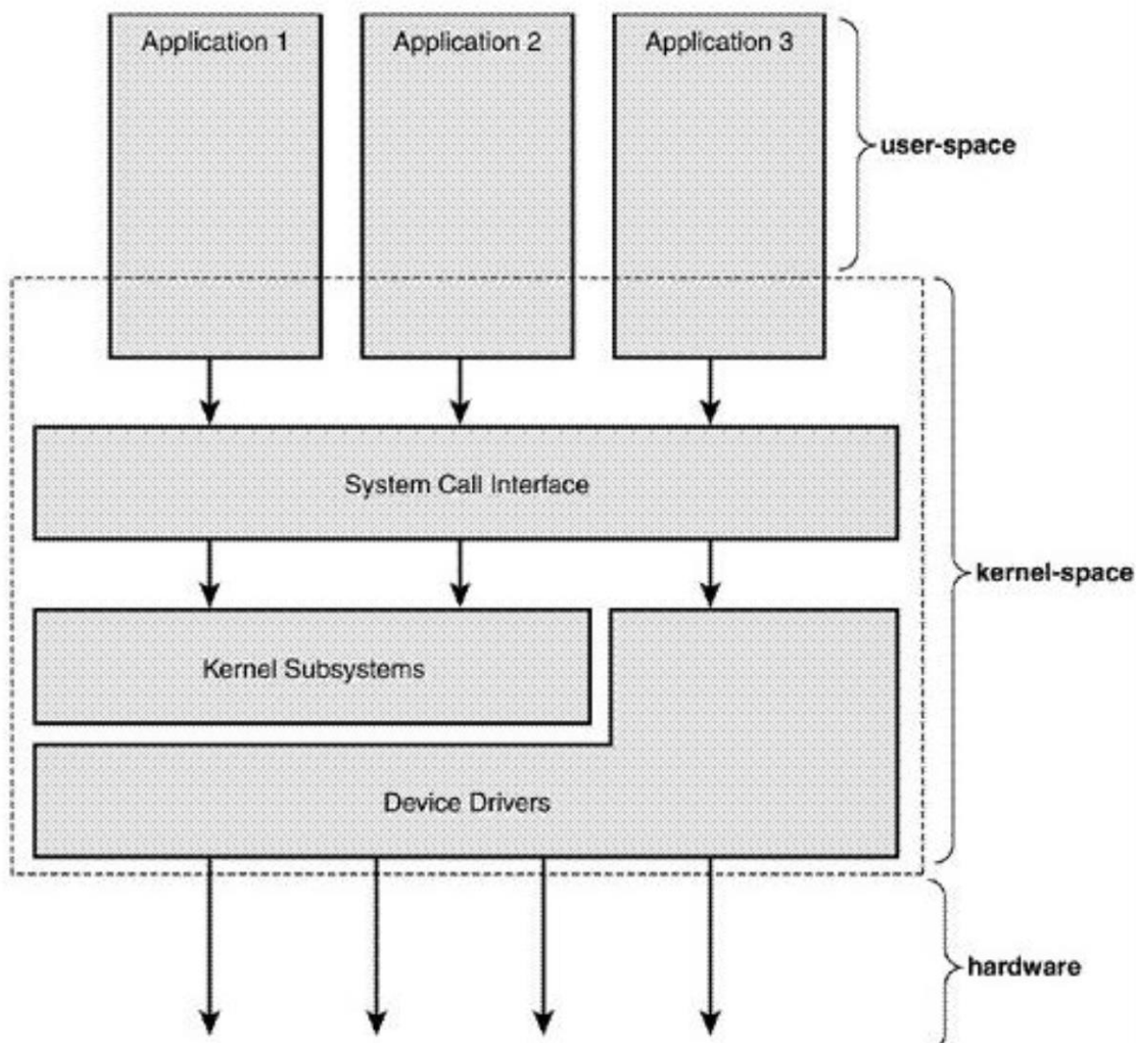
- **SIGHUP 1** – отправляется процессу при закрытии контролирующего его терминала.
- **SIGINT 2** – отправляется процессу контролирующим его терминалом, если пользователь прерывает работу процесса клавишами **Ctrl+C**.
- **SIGQUIT 3** – отправляется процессу, если пользователь посылает сигнал выхода из программы **Ctrl+D**.
- **SIGKILL 9** – этот сигнал немедленно завершает (убивает) процесс без выполнения любых операций очистки файлов, логов и т.д.
- **SIGTERM 15** – это сигнал завершения работы программы (он по умолчанию отправляется командой *kill*).
- **SIGTSTP 20** – отправляется процессу контролирующим его терминалом с запросом на остановку (*terminal stop*); инициируется при нажатии **Ctrl+Z**.

Все сигналы условно делятся на три класса:

- системные сигналы (ошибки аппаратуры, системные и т.д.);
- сигналы от устройств;
- сигналы, определённые пользователем.

Системные вызовы (syscall)

Архитектура *Linux* включает в себя пространство пользователя, ядра и устройств. Схематически это выглядит следующим образом:



Syscall (системный вызов) — своего рода *API* для обращения пользовательских команд и приложений к функциям ядра. Для чего нужна эта прослойка и почему нельзя напрямую обращаться в ядро? Причин, собственно, всего 3:

- **Простота.** Не нужно для каждой программы писать взаимодействие с железом. Просто представьте, если код условного отображения на экран `Hello world` от одной коротенькой строчки вдруг разрастётся до нескольких тысяч. А ведь надо же прописать взаимодействие с жёстким диском, отображением и прочие важные вещи);
- **Безопасность и стабильность.** Распределение ресурсов между ОС и программами пользователя (чтоб не выстрелить себе в ногу, предоставив все доступные ресурсы отображению `Hello world`);
- **Виртуальное представление процессов.**

Благо все [syscall](#) в *Linux* хорошо известны, задокументированы и пронумерованы. Так что рекомендую ознакомиться, кликнув по гиперссылке в прошлой строке.

Сокеты

Как вы помните, все в Linux является файлом — в том числе и сетевые сокеты. По большому счету, сокет является идентификатором сетевого соединения. При этом запись в сокет превращает записанное в сетевой пакет. При получении пакетов мы, соответственно, читаем информацию из сокета.

А где взять эти сокеты? Для этого есть специальный системный вызов — `socket()`. Раньше применяли системный вызов `socketcall()` в качестве общей точки входа в ядро для всех работ с сокетами, однако в архитектурах ARM и x86-64 этот системный вызов отсутствует, и теперь используют `socket()`.

`socket()` возвращает файловый дескриптор сокета и позволяет с помощью вызовов `send()` и `recv()` получать и отправлять информацию. Существует огромное количество видов сокетов, но мы остановимся на нескольких основных. Их еще называют «доменами».

Полный список доменов, которые поддерживает ваша система, можно посмотреть в терминале, введя команду: `man socket`

Первый и самый простой — **Internet Socket (домен AF_INET)**.

Эти сокеты используют для удаленного взаимодействия с программами в привычной нам манере. Внутри домена AF_INET есть два основных типа:

- SOCK_STREAM;
- SOCK_DGRAM.

Datagram сокеты иногда называют connectionless. SOCK_STREAM использует протокол TCP, а SOCK_DGRAM — протокол UDP. Тут важно понимать, что SOCK_STREAM и SOCK_DGRAM функционируют полярно разными «техниками».

Тип сокета не привязан к домену сокета. То есть, тип сокета SOCK_STREAM можно использовать и в домене AF_INET, и в домене AF_UNIX. Если вы поймете смысл домена и типа, то путаницы возникать не будет.

Если прямо совсем упрощать, то домен сокета, отвечает на вопрос «с кем?», а тип сокета — отвечает на вопрос «каким образом?».

Сокеты нужны не только для нас, как конечных пользователей ОС. Они, конечно же, используются без преувеличения практически всеми утилитами в Linux. Тип сокета SOCK_STREAM в домене AF_INET, к примеру, используется такими утилитами как ssh, telnet.

Ну и второй (он же последний) домен, который мы разберем, — **AF_UNIX**. Этот домен — «оптимизация» домена AF_INET. Представьте ситуацию: вы хотите отправить посылку в другой город. В таком случае вы ее упакуете в коробку, закрутите в пупырчатую пленку и напишете адрес получателя.

А теперь следующая ситуация — вам необходимо передать посылку своей бабушке, которая находится в соседней комнате. Очевидно, в этом случае вы просто пойдёте и передадите то, что необходимо, пропуская шаги упаковки и распаковки.

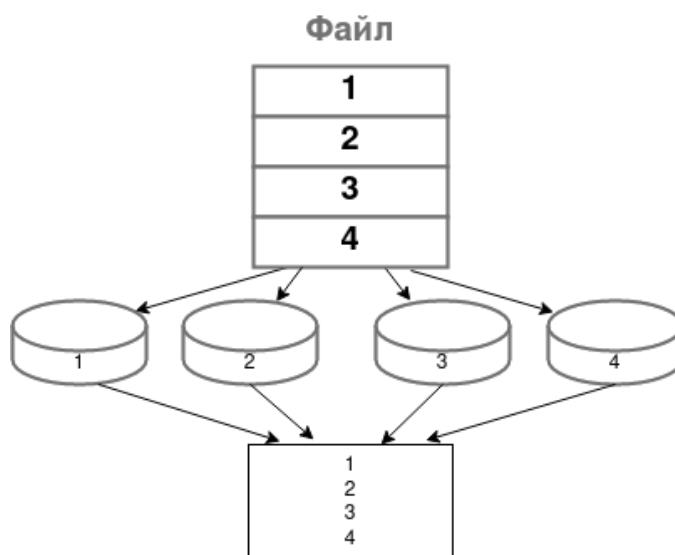
Описанные выше ситуации и есть то, что отличает домен AF_INET от домена AF_UNIX. Последний функционирует на уровне локальной системы, поэтому все «сообщения» не покидают ваше устройство, и в этом домене нет необходимости проводить ряд действий, которые проводит AF_INET при отправке пакетов за пределы локального устройства.

Дисковая и файловые подсистемы

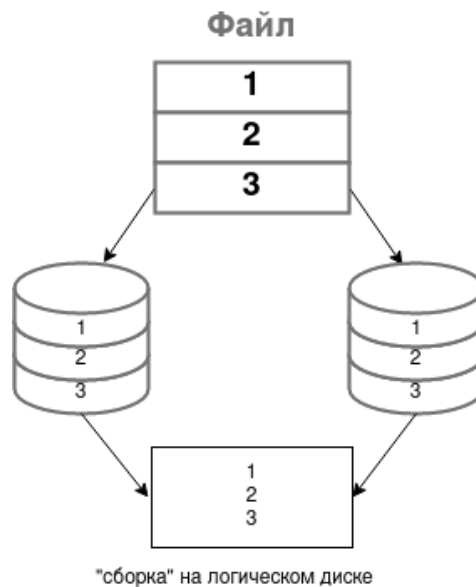
Дисковой подсистемой компьютера принято называть устройства, которые используют для повседневного хранения и считывания данных. Это *HDD*, *SSD*, *CD*, *USB Flash drive* и т.д.

Диски можно объединять в массивы — *RAID*. RAID-массивы были разработаны в целях повышения надёжности хранения данных, увеличения скорости работы с дисками и для обеспечения возможности объединения нескольких дисков в один большой диск. Разные типы *RAID* решают разные задачи хранения.

RAID 0 (Stripe). В этом режиме файл разбивается на одинаковые части и записывается на диски одновременно. За счёт этого достигается максимальная производительность, однако цена ей — ненадёжность. В случае, если хотя бы один диск выйдет из строя, данные в таком массиве «посыпятся», и вряд ли у вас получится их нормально восстановить.



RAID 1 (Mirror). В таком режиме файл записывается сразу на два диска. Выигрыша в производительности почти нет (разве что немного в скорости чтения), однако это компенсируется надёжностью. В случае выхода из строя одного диска, на втором будет его полноценная «копия».



RAID 5. Это массив, также использующий распределенное хранение данных, как и *RAID 0* + распределённое хранение кодов четности для восстановления данных при сбоях. Возможно как одновременное чтение, так и запись.



Плюсом этого варианта является то, что доступная для пользователя ёмкость массива уменьшается на ёмкость лишь одного диска, хотя надёжность хранения данных ниже, чем у *RAID 1*. По сути, является компромиссом между *RAID 0* и *RAID 1*, обеспечивая достаточно высокую скорость работы при неплохой надёжности хранения данных. При отказе одного диска из массива данные могут быть восстановлены без потерь в автоматическом режиме. Минимальное количество дисков для такого массива — 3.

Есть ещё несколько вариантов *RAID* массивов, но, как правило, это различные комбинации массивов 0, 1 и 5.

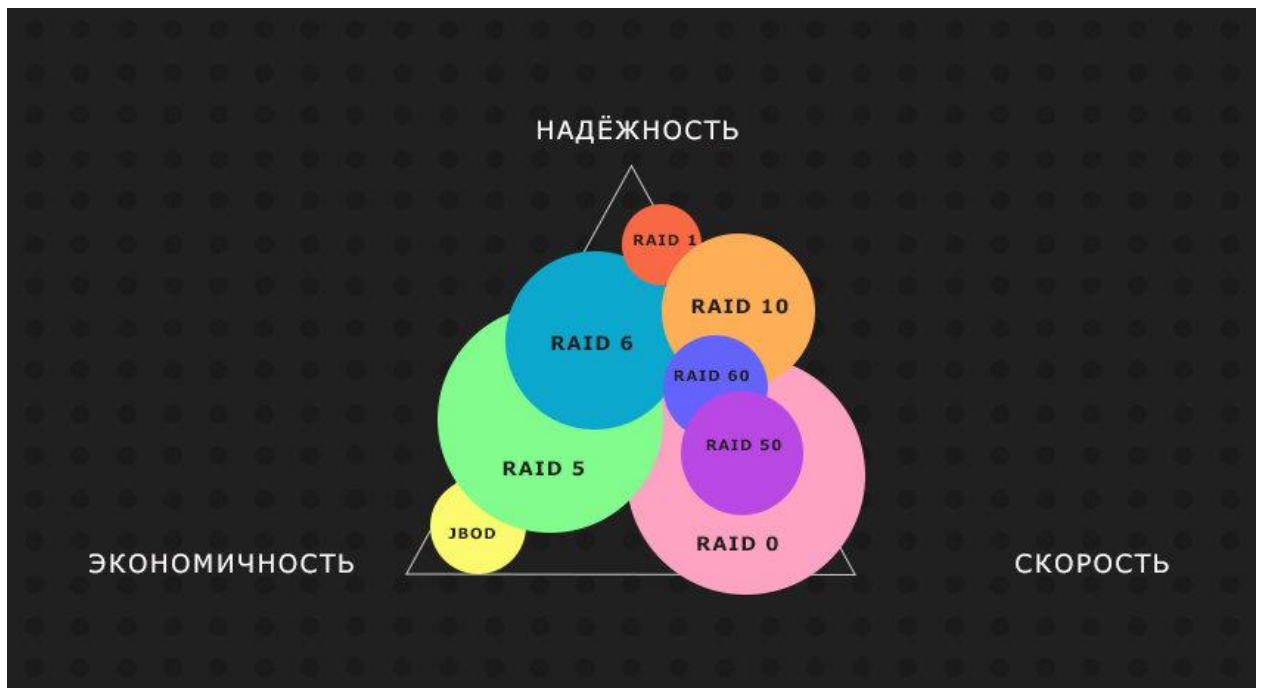


Таблица разделов

Существует всего 2 таблицы разделов — **MBR** (*master boot records*) и **GPT** (*GUID Partition table*).

MBR — это устаревший формат таблицы разделов, однако используется до сих пор. Занимает первые 512 байт в самом начале диска. В этих 512 байтах содержится информация о логических и расширенных (*logical & extended*) разделах, исполняемый код поиска загрузчика ОС на разделах (в случае с *Linux* — ищется *GRUB*).

GPT более современный стандарт. В этом разделе по-другому осуществляется адресация (блоками памяти, в то время как в **MBR** адрес просто состоял из 3-х значений: головка, цилиндр, и сектор). В **GPT** содержится только таблица разделов, так как остальные функции предполагают выполнение на других компонентах (например, на *EFI*). В целях «защиты от дурака», для предотвращения затирания утилитами всех блоков, в самом первом блоке *LBA0* всё же содержится **MBR**, а с *LBA1* уже начинается сам **GPT**.

Сравнительная таблица **GPT** и **MBR**:

	<i>MBR</i>	<i>GPT</i>
Поддержка объёма дисков	До 2 ТБ	До 9 Зеттабайт (очень много)
Количество разделов	До 4-х	Больше 4-х
Адресация	3 цифры	Блочная <i>LBA</i>
Расположение служебной информации	В начале диска	В начале и в конце диска
Контроль изменения таблицы разделов	Нет	Осуществляется с помощью хеша
Поддержка <i>Unicode</i>	Нет	Есть

Файловая подсистема

Чтобы до конца понять суть файловых подсистем в *Linux*, надо сначала осознать (и принять) его главную парадигму, которую я уже упоминал ранее, а именно «**всё есть файл**» (процессы, железо, периферия, вообще всё).

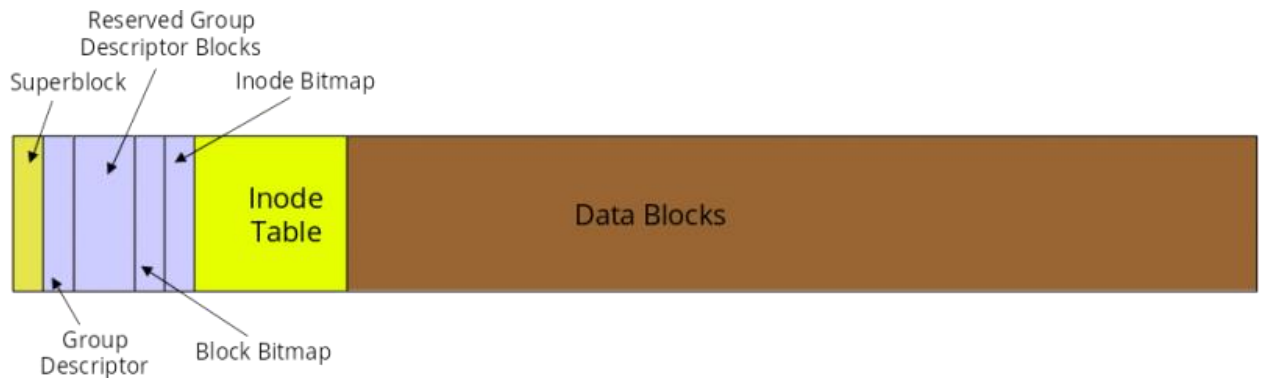
Файловая система — это структура, с помощью которой операционная система предоставляет пользователям и процессам ресурсы внешней памяти.

Операционная система *Linux* поддерживает работу с *FAT16* и *FAT32* в полном объёме, а работу с *NTFS* обеспечивает только в режиме чтения данных (начиная с версии 5.15 в ядро *Linux* добавлена поддержка *NTFS*). Кроме этого в данной операционной системе широко используются следующие файловые системы:

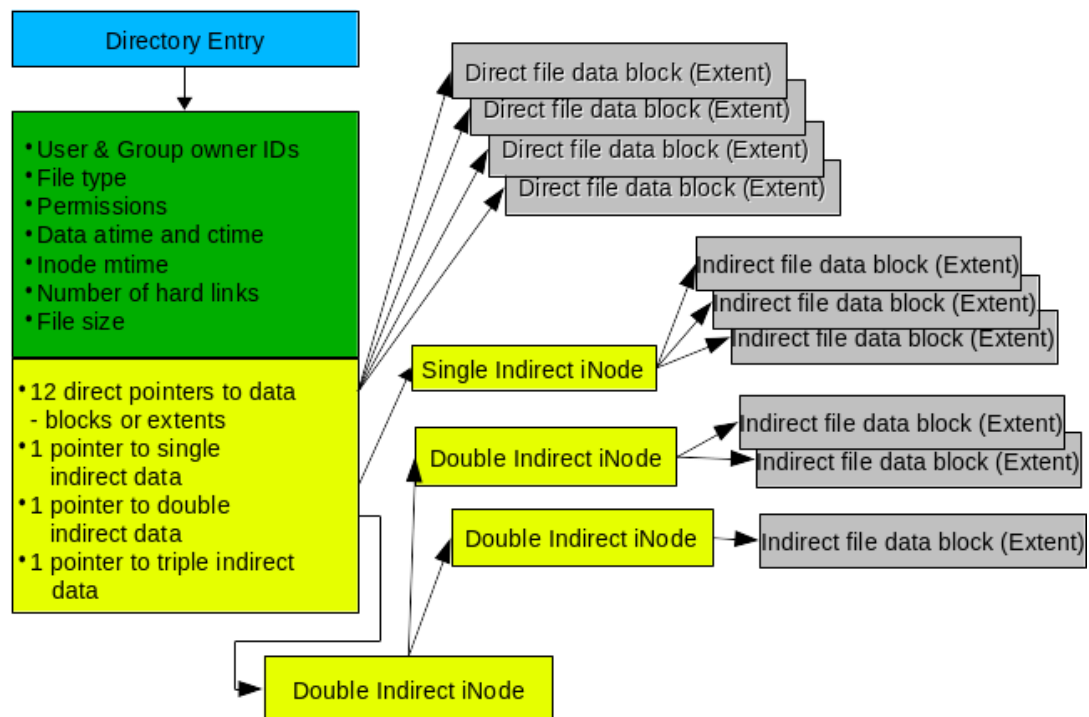
- *ext2* — устаревшая ФС, с высокой производительностью (почти на уровне *ext4*), только не журналируемая;
- *ext3* — журналируемая файловая система, полностью совместимая с *Ext2*, но менее производительная. Также устарела и почти не используется;
- *ext4* — современная журналируемая ФС. На сегодняшний день, самая широко используемая в ОС *Linux*;
- *UnionFS* — вспомогательная ФС, которая используется при контейнеризации. Это нежурналируемая ФС, которая может монтироваться каскадно (накладывая друг на друга) другие ФС. Благодаря такому монтированию файлы и каталоги способны «перекрываться».

Журналируемая файловая система — это такая система, в которой ведётся лог (журнал). В него записываются изменения файлов и метаданных перед внесением этих изменений (т.е. сначала пишем в лог то, что хотели сделать с файлом, а уже потом делаем), а после внесения изменений удаляет записи о них (то есть внесли изменения в файл, удалили запись в лог). Журналирование нужно в целях обеспечения проверки целостности и снижения шанса потери данных.

Давайте чуть подробнее остановимся на самой популярной на сегодняшний день файловой системе для *Linux* — **ext4**. Эта файловая система хранит данные блоками, каждый по 4096 байта. В начале раздела расположен так называемый суперблок, в котором располагаются метаданные всей файловой системы (физическая адресация блоков, их структура), групповой дескриптор, *bitmap* (отслеживает используемые и свободные блоки системы) и таблица *Inode* (становится душно, не правда ли?).



Inode (индексный дескриптор) — это структура данных, в которой хранятся метаданные файла и перечислены блоки с данными файла. *Inode* — ключевой компонент метаданных в *ext* системах. На рисунке далее отображена взаимосвязь между *inodes* и данными на жёстком диске. По факту на схеме отображён *inode* для одного файла, который сильно фрагментирован (физически располагается на разных местах магнитного блинчика). Файловые системы типа *ext* пытаются размещать данные таким образом, чтобы фрагментация была как можно меньше (чтобы файлы не были размазаны, как сгущёнка по тарту).



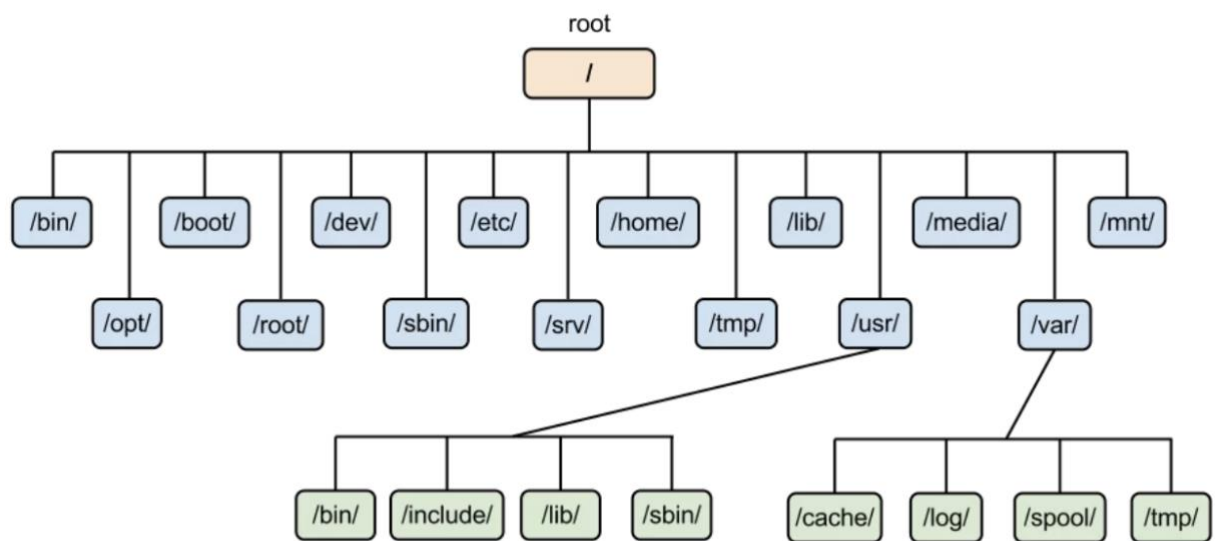
inode содержит в себе:

- метаданные о файле;
- тип файла;
- разрешения файла;
- размер;
- 15 указателей, описывающих расположение и длину блоков или экстендов (экстенды — это непрерывная область носителя информации).

В случае, если заполнятся все *inodes*, то даже при свободном месте на дисках у вас не выйдет записать каких-либо данных.

Структура каталогов

Чтобы более-менее ориентироваться в структуре каталогов, необходимо не один месяц поработать с ОС Linux. Тем не менее полезно знать, какой каталог для чего предназначен. Для этого приводим иерархию основных каталогов с их назначением. Она не привязана к какому-либо дистрибутиву, вы встретите такую иерархию практически в любой версии семейства Linux.



Каталог	Назначение
/	Root-каталог. Содержит в себе всю иерархию системы.
/bin	Здесь находятся двоичные исполняемые файлы. Основные общие команды, хранящиеся отдельно от других программ в системе (прим.: <code>pwd</code> , <code>ls</code> , <code>cat</code> , <code>ps</code>).
/boot	Тут расположены файлы, используемые для загрузки системы (образ <code>initrd</code> , ядро <code>vmlinuz</code>).
/dev	В этой директории располагаются файлы устройств (драйверов). С помощью этих файлов можно взаимодействовать с устройствами. К примеру, если это жесткий диск, можно подключить его к файловой системе. В файл принтера же можно написать напрямую и отправить задание на печать.
/etc	В этой директории находятся файлы конфигураций программ. Эти файлы позволяют настраивать системы, сервисы, скрипты системных демонов.
/home	Каталог, аналогичный каталогу <code>Users</code> в <code>Windows</code> . Содержит домашние каталоги учетных записей пользователей (кроме <code>root</code>). При создании нового пользователя здесь создается одноименный каталог с аналогичным именем и хранит личные файлы этого пользователя.
/lib	Содержит системные библиотеки, с которыми работают программы и модули ядра.

<code>/lost+found</code>	Содержит файлы, восстановленные после сбоя работы системы. Система проведет проверку после сбоя и найденные файлы можно будет посмотреть в данном каталоге.
<code>/media</code>	Точка монтирования внешних носителей. Например, когда вы вставляете диск в дисковод, он будет автоматически смонтирован в директорию <code>/media/cdrom</code> .
<code>/mnt</code>	Точка временного монтирования. Файловые системы подключаемых устройств обычно монтируются в этот каталог для временного использования.
<code>/opt</code>	Тут расположены дополнительные (необязательные) приложения. Такие программы обычно не подчиняются принятой иерархии и хранят свои файлы в одном подкаталоге (бинарные, библиотеки, конфигурации).
<code>/proc</code>	Содержит файлы, хранящие информацию о запущенных процессах и о состоянии ядра ОС.
<code>/root</code>	Директория, которая содержит файлы и личные настройки суперпользователя.

/run	Содержит файлы состояния приложений. Например, PID-файлы или UNIX-сокеты.
/sbin	Аналогично /bin содержит бинарные файлы. Утилиты нужны для настройки и администрирования системы суперпользователем.
/srv	Содержит файлы сервисов, предоставляемых сервером (прим.: FTP или Apache HTTP).
/sys	Содержит данные непосредственно о системе. Тут можно узнать информацию о ядре, драйверах и устройствах.
/tmp	Содержит временные файлы. Такие файлы доступны всем пользователям на чтение и запись. Этот каталог очищается при перезагрузке.
/usr	Содержит пользовательские приложения и утилиты второго уровня, используемые пользователями, а не системой. Содержимое доступно только для чтения (кроме root). Каталог имеет вторичную иерархию и похож на корневой.
/var	Содержит переменные файлы. Имеет подкаталоги, отвечающие за отдельные переменные. Например, логи будут храниться в /var/log, кэш в /var/cache, очереди заданий в /var/spool/ и так далее.

Специальные файловые системы

Исходя из знакомой нам уже парадигмы «всё есть файл, в *Linux* есть специальные файловые системы. Вот от некоторые из них:

- **tmpfs** — здесь можно взаимодействовать с оперативной памятью, т.е. писать, читать и исполнять файлы прямо из оперативной памяти (естественно после выключения или перезагрузки оттуда всё будет стёрто);
- **procfs** — здесь хранится информация процессах и о ядре. В системе примонтирована (по умолчанию) в */proc*;
- **sysfs** — здесь содержится информация о драйверах и железе.

Основные функции файловой системы

Самая главная функция любой файловой системы заключается в **предоставлении «места» для файлов**, программ, ОС и пр. На это вы можете возразить, дескать мол можно же всё содержать в оперативной памяти, ведь она гораздо производительнее любого файлового хранилища...и будете отчасти правы. Однако тут есть один важный нюанс, почему для таких целей не используют оперативную память — это зависимость данных от внешней

энергии. Когда прекращается подача электричества, то данные в оперативной памяти бесследно пропадают, а вот на файловых системах нет (шах и мат ОЗУ!).

Когда речь заходит о файловой системе, данные необходимо организованно хранить (а то, кому ж нужен склад макретплейса без персонала?), то есть **обеспечивать пространство имён**. Таким образом, данные будут иметь своё название, структуру и тип (не будем же мы там на складе самотыки в одну кучу с детский питанием бросать, правда?).

Итак, мы сложили данные, организовали их по полочкам, но вот как же ориентироваться в таком огромном пространстве? С этой задачей помогут справиться **метаданные** (возвращаясь к аналогии со складом — это его пожарная схема). В метаданные записывается структура данных для создания «дерева» каталогов, структура свободных и занятых блоков на диске, информация о файлах (размер файла, время создания и пр.), логических томах на диске и описание файловых систем. Метаданные хранятся отдельно от самих файлов (по аналогии со складом — удобно, когда пожарная схема расположена прямо у входа на склад и мы точно знаем куда бежать в случае возгорания).

Вот некоторые утилиты для работы с файловыми системами:

Утилита	Применение
<i>mkfs</i>	Создать файловую систему
<i>gparted</i>	Управление разделами на диске
<i>e2label</i>	Изменить метку раздела с файловой системой <i>ext</i>
<i>tune2fs</i>	Изменить файловую систему <i>Linux</i> , настроить её параметры
<i>fsck</i>	Проверка файловой системы <i>Linux</i> и исправление найденных ошибок
<i>resize2fs</i>	Изменить размер раздела с файловой системой
<i>fdisk</i>	Разметка диска
<i>mount</i>	Монтирование файловых систем вручную
<i>lshw</i>	Получить информацию о подключенных дисках
<i>parted</i>	Управление разделами на диске

Для подробного изучения ключей каждой из команд, следует воспользоваться командой `man имя_команды_для_изучения` (например, `man lshw`).

Аудит

В операционной системе происходит множество процессов, которые незаметны для обычного пользователя. Более того, когда одним устройством пользуются несколько человек (и даже необязательно человек — это могут быть локальные скрипты или удаленные службы) под разными аккаунтами, количество событий, которые происходят в системе, разрастается до огромных размеров. Чтобы контролировать их, проводить аудит операционной системы и расследовать инциденты в ОС Linux, используют утилиту `audit`.

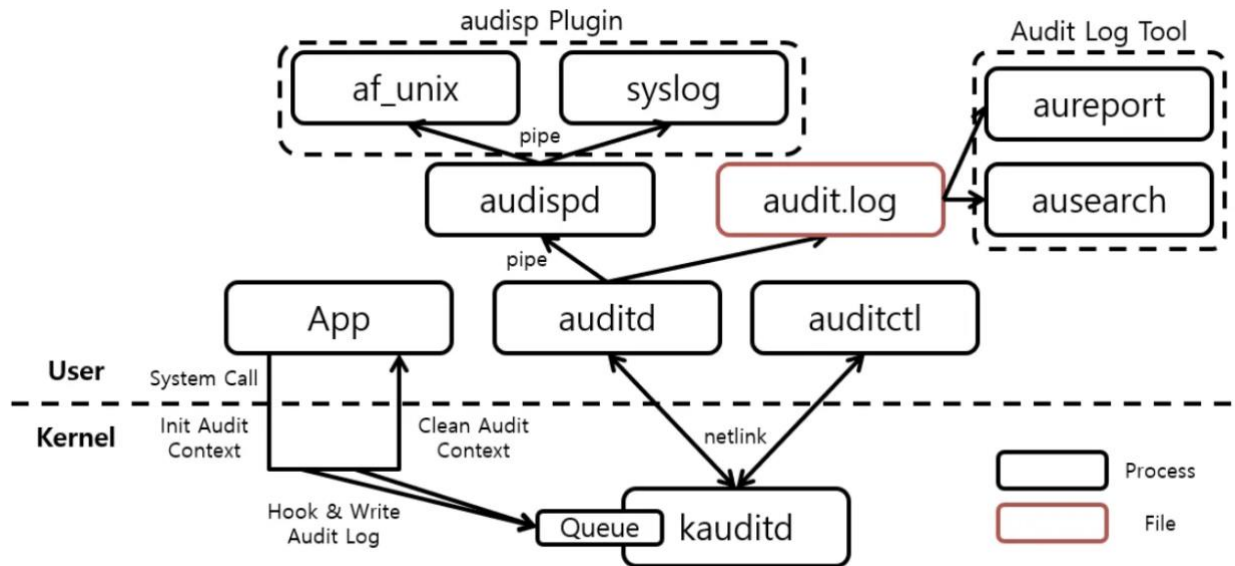
Кейсы использования audit достаточно многочисленны. В чём же заключается функция audit?



CAPP — это набор правил, который определяет необходимый функционал для построения безопасных информационных систем.

По большому счету, в ядро Linux уже встроены компоненты для мониторинга событий системы. Однако для взаимодействия с этими компонентами нам необходим сервис на уровне user-space — таким сервисом является auditd.

Сложным техническим языком (языком схем) принцип работы аудита в Linux можно описать так (более подробно поговорим о нём в главе про логи).



```
sudo apt install auditd
```

При работе с аудитом есть несколько основных компонентов, с которыми можно взаимодействовать:

- `/etc/audit/auditd.conf` — файл конфигурации демона, настраивает то, как будет происходить логирование;

- `/etc/audit/audit.rules` — файл конфигурации аудита, настраивает то, что будет логироваться;
- `/var/log/audit/audit.log` — файл логов, куда собирается вся информация.

При установке демона `auditd`, он автоматически запускается, однако его всегда можно запустить/перезапустить/остановить командой:

```
sudo systemctl start/restart/stop auditd
```

Будьте внимательны, по умолчанию в файле `/etc/audit/audit.rules` прописана опция `-D`. Это значит, что при перезапуске демона `auditd` у вас произойдет очистка логов.

Наибольшую сложность из себя представляет написание правил для сбора данных в файле `audit.rules`. Схема написания правил здесь достаточно схожа с написанием правил для популярного средства защиты информации — IDS. Вы еще не раз с ним столкнетесь при работе в сфере ИБ. Аудит в Linux — это первая ступень к пониманию принципов работы таких систем как SIEM, IDS, IPS.

Чтобы управлять аудитом, используют команду `auditctl`. Она и позволяет добавлять новые параметры и правила в `audit.rules`.

Давайте разберемся с основными опциями:

Опция	Действие
<code>-D</code>	Удалить все правила и логи
<code>-b 500</code>	Задаёт буфер для аудита
<code>-e</code>	Включить/выключить аудит (0 — выключить, 1 — включить)
<code>-p</code>	Разрешения (rwx)
<code>-w</code>	Мониторить файл/директорию
<code>-f</code>	Действие на невозможность обработки правила. 0 — ничего не делать, 2 — максимальная важность
<code>-k</code>	Фильтр ключевых слов

Например, включаем аудит и сразу увеличиваем буфер логов до 500:

```
auditctl -e 1
auditctl -b 500
```

Начинаем расписывать правила.

Поставим мониторинг на директорию `/etc/passwd` так, чтобы при любом изменении атрибутов, чтении, записи или исполнении файлов в этой директории, мы получали об этом информацию. Опцией `-k` мы задаем как бы лейбл для данного действия, чтобы потом в логах смогли быстро его найти:

```
auditctl -w /etc/passwd -p rwax -k act_passwd
```

После того, как мы вписали правило, можно протестировать его работу. Один раз посмотрим содержимое файла командой `cat /etc/passwd`.

Команда `ausearch` — один из способов быстрого просмотра логов. Вводим:

```
ausearch -k act_passwd
```

И видим, что аудит увидел наше действие и залогировал его:

```
root@skill-VirtualBox:/etc/audit# ausearch -k act_passwd
----
time->Tue Dec 13 04:42:27 2022
type=CONFIG_CHANGE msg=audit(1670895747.062:115): auid=4294967295 ses=4294967295
op=add_rule key="act_passwd" list=4 res=1
----
time->Tue Dec 13 04:43:30 2022
type=PROCTITLE msg=audit(1670895810.438:116): proctitle=636174002F65746332F706173
737764
type=PATH msg=audit(1670895810.438:116): item=0 name="/etc/passwd" inode=1045875
dev=08:01 mode=0100644 ouid=0 ogid=0 rdev=00:00 nametype=NORMAL cap_fp=00000000
00000000 cap_fi=0000000000000000 cap_fe=0 cap_fver=0
type=CWD msg=audit(1670895810.438:116): cwd="/etc/audit"
type=SYSCALL msg=audit(1670895810.438:116): arch=c000003e syscall=2 success=yes
exit=3 a0=7ffcdba517b3 a1=0 a2=20000 a3=69d items=1 ppid=18018 pid=18858 auid=42
94967295 uid=0 gid=0 euid=0 suid=0 fsuid=0 egid=0 sgid=0 fsgid=0 tty=pts4 ses=42
94967295 comm="cat" exe="/bin/cat" key="act_passwd"
root@skill-VirtualBox:/etc/audit#
```

Напоминаю, что при перезапуске `auditd`, все правила удалятся. Чтобы наши правила сохранялись, нам необходимо сохранить их в файл `audit.rules` вручную, что мы и сделаем:

```
GNU nano 2.5.3      File: audit.rules      Modified
# This file contains the auditctl rules that are loaded
# whenever the audit daemon is started via the initscripts.
# The rules are simply the parameters that would be passed
# to auditctl.

# First rule - delete all
-D

# Increase the buffers to survive stress events.
# Make this bigger for busy systems
-b 320

# Feel free to add below this line. See auditctl man page
-w /etc/passwd -p wrax -k act_passwd
```

Таким образом, мы научились работать на базовом уровне с аудитом и можете настраивать мониторинг за папками/файлами в Linux.

Управление логическими томами (LVM)

LVM (*Logical Volume Management*) позволяет управлять дисковым пространством на разных физических носителях на уровне ОС, а именно:

- Объединять между собой тома, в том числе частично (например, 10 ГБ с одного диска и 20 ГБ с другого объединить в один том).
- Разбиение (один том на 30 ГБ на одном физическом носителе разбивается на 10 ГБ на один носитель и 20 ГБ на другой, при этом содержимое тома не меняется).

LVM может быть следующих видов:

- **PV** (*Physical Volume*) — физические тома (это могут быть разделы или целые «неразбитые» диски);
- **VG** (*Volume Group*) — группа томов (объединяем физические тома (**PV**) в группу, создаём единый диск, который будем дальше разбивать произвольным образом);
- **LV** (*Logical Volume*) — раздел нового «единого диска» или же Группы Томов.

Для простоты понимания рассмотрим пример. Допустим у нас имеется установленная и работающая под *Linux* машина с двумя *HDD*. На одном диске полностью установлена система, на другом ничего нет (он пуст от слова совсем). С помощью **LVM**, текущую систему можно «разбить» на 2 физических диска. При этом мы получим значительный выигрыш в производительности.

Для управления **LVM** используется утилита *lvm2*.

Управление пакетами

Программное обеспечение в *Linux*, как правило, устанавливается из репозитория с помощью пакетных менеджеров (также, к слову, обновляется и сама ОС). Давайте разберём новые термины по порядку.

Пакет — это некий архив, содержащий в себе файлы конфигов, бинарные файлы, данные о зависимостях, инструкции со списком действий (они же скрипты) для установки. Вот некоторые примеры расширений пакетов: *.deb*, *.rpm*.

Репозиторий (или же «репа») — централизованное «хранилище» пакетов, которое может быть внешним и внутренним. Репозитория существует капец как много, и перечислять все точно нет смысла, однако основные типы всё же стоит запомнить.

- *main* — основной репозиторий, в котором расположено основное, полностью свободно распространяемое ПО;
- *restricted* — проприетарное (от англ. *proprietary software*) программное обеспечение, официально поддерживаемое дистрибутивом;
- *universe* — свободно распространяемое ПО, которое поддерживается комьюнити;
- *contrib* — пакеты содержат компоненты свободного ПО (но являются самостоятельными, независимыми от *main*).
- *non-free* — пакеты, не удовлетворяющие принципам свободного ПО.

На самом деле, их больше, но их название и цели зависят сугубо от функций дистрибутива. Репозитории в дистрибутивах на основе Debian располагаются в */etc/apt/sources.list*.

Например, в Ubuntu (и его производных) есть репозитории PPA.

PPA (Personal Package Archive) — это персональный репозиторий разработчика конкретной программы, где он хранит пакеты своих программ, которые еще не включены в основной репозиторий дистрибутива.

Если нужного пакета нет в официальном репозитории, то необходимо добавить в список пользовательский репозиторий, после чего обновить пакетный менеджер.

Для добавления пользовательского репозитория используют команду `add-apt-repository`

Пример добавления репозитория с Telegram в Ubuntu:

```
sudo add-apt-repository -y ppa:atareao/telegram
```

Опция `-y` используется для указания конкретного пакета или репозитория. При использовании данной опции пакетный менеджер будет искать точное совпадение с указанным названием.

Пакетный менеджер — это, по сути, софт, который управляет установкой, обновлением, кешированием, загрузкой, удалением пакетов в ОС. Примеры таких пакетных менеджеров: *apt, pacman, rpm, yum*.

Самый распространённый и простой вариант установки ПО в *Linux* — это использование пакетного менеджера. Он сам сходит за вас в репозиторий, поищет необходимый пакет, скачает, установит зависимости (в зависимости от пакета, конечно же) и установит непосредственно сам пакет.

Давайте разберёмся, что же ещё умеет пакетный менеджер:

- Искать информацию о тех или иных пакетах используя локальный кэш. Да, у пакетного менеджера есть специальный **локальный кэш** (отрадно, что хоть у кого-то есть кэш) со списком того, что есть на репозитории. По данному кэшу можно оффлайн искать информацию и разумеется, нужно время от времени этот кэш обновлять.
- Обновлять тот или иной пакет, версию ядра, сам дистрибутив.
- Удалять пакеты, в том числе с их зависимостями.
- Скачивать пакеты без установки (например, если надо установить позже).
- Разрешать зависимости.
- Скачивать и устанавливать пакеты определённой версии (если таковые имеются в репозитории);
- Осуществлять поиск среди установленных и неустановленных пакетов.

Не всегда бывает так, что софт, который необходимо установить, бывает в репозиториях (коммерческое ПО там точно найти не получится). В таком случае, обычно, используется **dpkg** — специальная утилита для установки и удаления пакетов. *dpkg* отличается от пакетных менеджеров тем, что не умеет работать с репозиториями и разрешать зависимости.

А что спрашивается делать тем, кто сам по себе очень умный и умудрился даже написать свою прогу и теперь хочет собрать свой пакет? В таких случаях поможет утилита **make**. Эта утилита предназначена для преобразования (как правило, компиляции исходников) файлов в автоматическом режиме. Сам процесс того, как будет происходить преобразование, описывается в файлике *makefile* (если запустим *make*, то, в первую очередь, будет произведён поиск *makefile* для получения инструкций). Правила в *makefile* описываются следующим образом:

- что делаем;
- что нужно для достижения цели;
- какие команды нужны для достижения цели.

А что, если вы скачали где-то исходный код и теперь хотите его собрать (скомпилировать)? Тут тоже поможет *make*! Достаточно скачать исходный код и выполнить *make install* для сборки пакета. Ну а затем его уже можно установить с помощью *dpkg*.

Так как эта книга для хакеров (мамкиных, разумеется) вы наверняка знакомы с *Kali Linux* (это самый распространённый дистрибутив среди безопасников основанный на ядре Debian). Поэтому далее предлагаю именно на *Kali* разобрать примеры команд для пакетного менеджера *apt*.

Команда	Пояснение
<code>apt install</code>	Установка пакета из репозитория
<code>apt update</code>	Обновление данных о репозитории
<code>apt upgrade</code>	Обновление версий установленных пакетов (если доступно)
<code>apt search</code>	Поиск пакетов (в том числе поддерживаются регулярки)
<code>apt remove</code>	Удаление пакета
<code>apt autoremove</code>	Удаление пакета и его зависимостей

Репозитории в дистрибутиве *Kali* располагаются по следующему пути: `/etc/apt/sources.list`.

```
root@kali:/home/kali# cat /etc/apt/sources.list
#
# deb cdrom:[Kali GNU/Linux 2020.2rc1 _Kali-last-snapshot_ - Official amd64 DVD Binary-1 with firmware 20200505-14:58]/ kali-rolling contrib main non-free
# deb cdrom:[Kali GNU/Linux 2020.2rc1 _Kali-last-snapshot_ - Official amd64 DVD Binary-1 with firmware 20200505-14:58]/ kali-rolling contrib main non-free
deb http://http.kali.org/kali kali-rolling main non-free contrib
# deb-src http://http.kali.org/kali kali-rolling main non-free contrib
# This system was installed using small removable media
# (e.g. netinst, live or single CD). The matching "deb cdrom"
# entries were disabled at the end of the installation process.
# For information about how to configure apt package sources,
# see the sources.list(5) manual.
root@kali:/home/kali#
```

Пакетный менеджер *apt* будет обращаться к источникам, которые указаны в обозначенном файле. Принцип работы пакетных менеджеров одинаков для всех дистрибутивов, но могут (и поверьте мне, будут) различаться ключи, расположение самого файла с репозиториями и конечно пакетный менеджер.

Мы научились пользоваться стандартной пользовательской функцией ОС Linux, без которой нельзя протянуть в терминале и пяти минут. Управлять пакетами, репозиториями, устанавливать программы и утилиты — база для любого пользователя данной операционной системы.

Управление пользователями и привилегиями

Навык управление пользователями и привилегиями на бытовом уровне требуется достаточно редко, поскольку операционной системой в домашних условиях пользуется

один, максимум два человека. Однако в корпоративных сетях это число вырастает иногда до нескольких десятков человек. Плюс здесь применяется парадигма Zero Trust, при которой системный администратор на пару с ИБ-шником должны сделать всё, чтобы обычный пользователь не создал инцидент на ровном месте. Поэтому для человека, занимающегося информационной безопасностью, критически важно понимать, как происходит «разделение» доступа.

Модель AAA

Модель AAA (трипл-эй!) — это аутентификация (*authentication*), авторизация (*authorization*) и аккаунтинг (*accounting*). Данные процедуры отвечают за разграничение прав доступа, безопасность и учёт данных пользователя. Далее мы подробно рассмотрим каждый из них.

Аутентификация — процесс подтверждения подлинности (докажи, что ты тот, кем представился). Аутентификация будет успешна только в том случае, если субъект (тот, кто аутентифицируется) существует, и он представил корректный пароль (токен, отпечаток пальца или любой другой фактор, которым обладает субъект).

Авторизация — процесс подтверждения прав субъекта аутентификации (того, кто успешно аутентифицировался). С помощью авторизации можно гибко распределять права на что-либо внутри системы (например, доступ к файлам, разрешение на выполнение программ и так далее).

Самый простой пример здесь — обычные сайты. Мало кто знает, но большинство сайтов, особенно написанных на популярных фреймворках, имеют панели управления администратора. Они позволяют редактировать контент сайта, добавлять новые товары, менять цены и так далее. Это прямой пример «авторизации» — администратор авторизован на изменение определенного контента путем входа в администраторскую панель. Пользователь авторизован на просмотр страничек сайта путем входа на сайт.

Аккаунтинг — учет данных аутентифицированного пользователя. Сюда можно отнести ведение логов (журналов), подсчет сетевого трафика, мониторинг используемых вычислительных мощностей.

Для наглядности давайте начнём с примера использования модели AAA в *Linux*. После включения ПК нам, как правило, предлагается ввести имя пользователя и пароль — это процедура аутентификации. После ввода логина и пароля мы попадаем в графическую оболочку (либо консоль, если GUI не доставили), то есть успешно проходим аутентификацию. Далее пробуем открыть файл `/etc/passwd` в системе с помощью утилиты `cat` или любого другого текстового редактора и нам это удаётся, т.к. (по умолчанию) права на чтение есть у всех пользователей системы, то есть, по сути, успешно проходим процедуру авторизации.

```
kali@kali:~$ ls -la /etc |grep passwd
-rw-r--r-- 1 root root 3107 Feb 18 12:43 passwd
-rw-r--r-- 1 root root 3111 May 8 2020 passwd-
```

Пробуем получить доступ к `/var/log/auth.log` и получаем ошибку о том, что у нас не хватает прав, то есть разрешения данного файла не позволяют нам его прочитать (не проходим авторизацию).

```
kali@kali:~$ sudo ls -ls /var/log/ |grep auth
84 -rw-r----- 1 root adm 78989 Feb 26 06:53 auth.log
```

Повышаем права до `root` с помощью утилиты `sudo` и читаем лог.

```
Feb 26 06:46:41 kali sudo: pam_unix(sudo:session): session closed for user root
Feb 26 06:49:56 kali sudo:    kali : TTY=pts/0 ; PWD=/home/kali ; USER=root ; COM
MAND=/usr/bin/cat /var/log/auth.log
Feb 26 06:49:56 kali sudo: pam_unix(sudo:session): session opened for user root(ui
d=0) by (uid=1000)
```

Видим, что наши действия записываются, то есть ведётся так называемый аккаунтинг (учёт наших действий в системе).

Атрибуты и права файлов

Окей. Теперь давайте разберёмся, какими же атрибутами обладают файлы в *Linux*.

- Утилита `ls` предназначена для отображения содержимого папки,
- Ключ `-l` отвечает за отображение разрешений,
- Ключ `-a` отображает скрытые элементы.

Разрешения могут быть `drwxrwxrwx+`. Давайте для простоты восприятия, разделим пробелами: `d rwx rwx rwx +` (на самом деле, это ещё можно записать как `777` или `111111111`, если представлять в виде бинарного кода)

- `d` — технически, не является частью разрешений. Этот символ указывает, является ли файл директорией. Если файл не является директорией, то проставляется символ `-`. Ещё может приниматься значение `l` (link, она же ссылка на файл).
- `rwx` — первые три символа указывают на разрешения владельца файла.
- `rwx` — вторые три символа указывают на разрешения группы — владельца файла.
- `rwx` — третьи три символа указывают на разрешения для всех остальных (все, кто не является владельцем и не входит в группу владельца).
- `+` — указывает, применяется ли к файлу (папке) альтернативный метод доступа. Может также принимать значение «.» в случае, если на файле действует специальный режим доступа (например, с помощью Access control list);

rwx (read write eXecut). Первая буква означает права на чтение, вторая — на запись, третья — выполнение. Ниже таблица с пояснением, как и какое разрешение может принимать файл или директория и как это будет сказываться на доступе:

	Значение	Применение к файлу	Применение к директории
r (read — чтение) (4)	—	Файл не может быть прочтён.	Не может быть отображено содержимое директории.
	r	Файл может быть прочтён.	Может быть отображено содержимое директории.
w (write — запись) (2)	w	Файл может быть изменён.	Содержимое директории может быть изменено (требует также прав на выполнение, иначе содержимое нельзя будет изменить).
	—	Файл не может быть изменён.	Содержимое директории нельзя изменить, также нельзя сделать её CWD (текущая рабочая директория; например, с помощью команды <code>cd</code>).
x (eXecute — выполнение) (1)	—	Файл не может быть выполнен.	Нельзя сделать директорию CWD (текущая рабочая директория; например, с помощью команды <code>cd</code>).
	x	Файл может быть выполнен.	Можно перейти в директорию и сделать её CWD (<i>current working directory</i>), например, с помощью <code>cd</code> .
	s	Бит <i>setuid</i> или <i>setgid</i> . В случае, если этот бит установлен в первых трёх символах прав файла — то это <i>setuid</i> , если во вторых — <i>setgid</i> . Установление этого бита позволяет выполнять файл от имени пользователя-владельца файла (<i>setuid</i>) или от имени группы-владельца файла (<i>setgid</i>). В случае применения к директории, все новые созданные объекты внутри директории будут наследовать бит <i>setuid</i> (<i>setgid</i>), но не применяется к уже существующим объектам внутри папки.	
	S	Тот же <i>setuid</i> или <i>setgid</i> , но в случае, если не установлены разрешения на выполнения, то есть x. Очень редкое явление (никогда в реальной жизни не встречал).	
	t	<i>Sticky bit</i> может быть только для всех остальных пользователей . Установленный <i>sticky</i> бит означает, что только пользователь — владелец файла (директории) может его удалить, при этом все остальные пользователи могут редактировать файл со <i>sticky bit</i> . Например, директория <code>/tmp</code> .	
	T	<i>Sticky bit</i> без разрешения на выполнение (никогда в реальной жизни не встречал).	

■

Расширенные атрибуты

Атрибуты файла не зависят от того, кто является владельцем файла, и кто с ним работает. Выставление атрибутов помогает обеспечить дополнительный уровень защиты файлов или директорий на сервере. Атрибуты можно посмотреть с помощью утилиты *lsattr*, а изменить через *chattr*.

Основные атрибуты:

- **A** — этот атрибут гарантирует, что время доступа к файлу не изменяется. Обычно каждый раз, когда файл открывается, время доступа к файлу должно быть записано в метаданные файла. Это отрицательно влияет на производительность; поэтому для файлов, к которым осуществляется регулярный доступ, атрибут A можно использовать для отключения этой функции;
- **a** — этот атрибут позволяет дополнять файл, но не удалять его;
- **c** — если вы используете файловую систему, в которой поддерживается сжатие на уровне тома, этот атрибут файла гарантирует, что файл будет сжат при первом включении механизма сжатия;
- **D** — этот атрибут гарантирует, что изменения в файлах записываются на диск немедленно, а не в кеширование в первую очередь. Это полезный атрибут в важных файлах базы данных, позволяющий убедиться, что они не теряются между файловым кэшем и жёстким диском;
- **d** — этот атрибут гарантирует, что файл не будет сохранён в резервных копиях, где используется утилита дампа;

- **e** — означает, что файл использует экстенды для соотношения блоков на диске (этот атрибут нельзя удалить используя *chattr*);
- **I** — этот атрибут включает индексирование для каталога, в котором он включен. Это обеспечивает более быстрый доступ к файлам для примитивных файловых систем, таких как *ext3*;
- **i** — этот атрибут делает файл неизменным;
- **j** — этот атрибут гарантирует, что в файловой системе *ext3* файл сначала записывается в журнал, а затем — в блоки данных на жёстком диске;
- **s** — перезаписать блоки, в которых файл был сохранён, на 0 с после удаления файла. Это гарантирует, что восстановление файла невозможно после того, как он был удалён;
- **u** — этот атрибут сохраняет информацию об удалении. Это позволяет разрабатывать утилиту, которая работает с этой информацией для спасения удалённых файлов.

```

root@kali:~# lsattr /
_____e_____ /boot
_____e_____ /opt
  
```

Самом собой на самом деле расширенных атрибутов файлов намного больше. Более подробно они описаны в мануале к команде *chattr* (*man chattr*). Но помните, что расширенные атрибуты файлов следует использовать с осторожностью: это редкий и не самый простой инструмент для управления файлами в *Linux*.

Пользователи и группы

Пользователи и группы в *Linux* используются для обеспечения контроля и обеспечения доступа к файловой системе, директориям и периферийным устройствам. У каждого пользователя и группы есть обязательный атрибут — *id*. Для пользователей это *uid*, для групп — *gid*.

Обычно, *uid* назначается новым пользователям (в зависимости от дистрибутива) более 500 или более 1000 (если таковой не указан явно). Все пользователи, у которых *uid* меньше 1000 (или 500) считаются сервисными (например, часто веб-сервер имеет *uid* 33 и имя пользователя *www-data*). Особый случай — это *uid* 0. Пользователь с этим *uid* имеет наивысшие привилегии в системе (да-да понимаю ваш шок от того, что не всегда *root* — это пользователь с наивысшими привилегиями в системе).

Каждый пользователь в системе *Linux* должен состоять в какой-либо группе. Эта группа может быть даже из 1 пользователя (например, по умолчанию, при создании нового пользователя в *debian-based* дистрибутивах создаётся группа с одноимённым названием, в которую и будет помещён новый пользователь, если это не указано явно).

Вся информация о пользователях, группах и их паролях находится в специальных файлах:

- `/etc/passwd` — общая информация о пользователях,
- `/etc/shadow` — информация о пользователях и их паролях (или их отсутствии),
- `/etc/group` — информация о принадлежности пользователей к группам.

```
root@kali:~# ls -la /etc |grep " shadow$|group$|passwd$"  
-rw-r--r-- 1 root    root    1186 May  8  2020 group  
-rw-r--r-- 1 root    root    3107 Feb 18 12:43 passwd  
-rw-r----- 1 root    shadow 1639 May  8  2020 shadow
```

Далее давайте рассмотрим примеры этих файлов.

Начнём с `/etc/passwd`:

```
root@kali:~# cat /etc/passwd |grep "^root$|kali"  
root:x:0:0:root:/root:/bin/bash  
kali:x:1000:1000:Kali,,,:/home/kali:/bin/bash
```

В файле `/etc/passwd` через «:» перечислена следующая информация о пользователях через «:» `account:password:UID:GID:GECOS:directory:shell`

- `account` — имя пользователя (не может быть пустым);
- `password` — пароль пользователя в открытом виде (на момент написания финальной версии материала все пароли уже «переехали» из файла `/etc/passwd` в `/etc/shadow`);
- `UID` — *user id*;
- `GID` — *group id*;
- `GECOS` — комментарий, который может быть пустым;
- `directory` — домашняя директория пользователя, используется для создания переменной `$HOME`;
- `shell` — командная оболочка.

В файле `/etc/shadow` хранятся зашифрованные пароли пользователей. Вот пример для двух пользователей (`root` и `kali`):

```
root@kali:~# cat /etc/shadow |grep "^root$|kali"  
root:!:18390:0:99999:7:::  
kali:$6$8iEAYF4pzDp5QJXH$00iURpImugngJWdAraUxBFFU76xovc6mwuQMe479uN28NesybQx0W7TiLxqrn99nJCfvISJtJiXgIjtTicF720:18390:0:99999:7:::
```

Формат `/etc/shadow`:

```
пользователь:хэш пароля:дни_после_смены_пароля_с_01.01.1970:  
дни_до_разрешения_смены_пароля:дни_жизни_пароля:  
дни_до_оповещения_пользователя_о_смене_пароля:  
дни_после_протухания_пароля_блокировка_аккаунта:  
дни_отключенного_пользователя:резерв
```

Пример файла */etc/group* для пользователей *root* и *kali*:

```
root@kali:~# cat /etc/group |grep "^root\|kali"
root:x:0:
cdrom:x:24:kali
floppy:x:25:kali
sudo:x:27:kali
audio:x:29:pulse,kali
dip:x:30:kali
video:x:44:kali
plugdev:x:46:kali
netdev:x:109:kali
bluetooth:x:117:kali
kali-trusted:x:124:
scanner:x:132:saned,kali
kali:x:1000:
```

Формат записей — :имя_группы:резерв:gid:пользователи_группы

Управление пользователями и группами может осуществляться как из графической оболочки, так и с помощью терминала. За создание пользователя отвечает утилита *useradd*, за редактирование — *usermod*, за изменение пароля — *passwd*, за удаление — *userdel*.

За создание группы отвечает утилита *groupadd*, за изменение группы — *groupmod*, а за удаление группы — *groupdel*.

Управление пользователями и группами

Процесс построения безопасной системы начинается с прав доступа. И если создавать пользователей и группы мы научились, то выдача прав через атрибуты — не панацея от бед с безопасностью.

В Linux существует механизм, который позволяет в более широком смысле управлять доступом не только к каталогам, но и к сервисам и командам. Имя этому механизму — *sudoers*.

Файл *sudoers* располагается в директории */etc* и позволяет накладывать «прослойку» на пользователей и группы. Давайте разберемся с основными кейсами, в которых пригодится файл *sudoers*,

Кейс 1. Исполнение всех команд от другого пользователя/группы

По умолчанию в файле уже есть запись для пользователя *root*:

```
#
# This file MUST be edited with the 'visudo' command as root.
#
# Please consider adding local content in /etc/sudoers.d/ instead of
# directly modifying this file.
#
# See the man page for details on how to write a sudoers file.
#
Defaults        env_reset
Defaults        mail_badpass
Defaults        secure_path="/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:$
# Host alias specification
# User alias specification
# Cmnd alias specification
# User privilege specification
root    ALL=(ALL:ALL) ALL
# Members of the admin group may gain root privileges
%admin   ALL=(ALL) ALL
# Allow members of group sudo to execute any command
%sudo   ALL=(ALL:ALL) ALL
# See sudoers(5) for more information on "#include" directives:
```

В разделе «User privilege specification» мы можем наблюдать, что пользователю root на всех хостах (первый ALL) можно выполнять любую команду (четвертый ALL) от лица любого пользователя любой группы (второй:третий ALL).

Кейс 2. Исполнение определенной команды от другого пользователя/группы

Здесь принцип точно такой же, как и в первом пункте, только вместо четвертого ALL мы указываем конкретную команду, которую пользователь может исполнять. Например, мы хотим, чтобы пользователь skill мог просматривать файл shadow с помощью cat от имени root пользователя:

```
skill ALL = (root) /usr/bin/cat /etc/shadow
```

Все эти функции применимы и к группам, однако группа обозначается со знаком процента. В остальном синтаксис записей идентичен.

Кейс 3. Исполнение команд без ввода пароля

Аналогично предыдущим синтаксам, мы можем дать пользователю или группе возможность исполнять команды без запроса пароля. Для этого используйте ключевое слово NOPASSWD:

```
skill ALL = (root) NOPASSWD : ALL
```

Точно так же можно разрешить выполнение без пароля определенной команды:

```
skill ALL = (root) NOPASSWD : /usr/sbin/apache2
```

Кейс 4. Алиасы

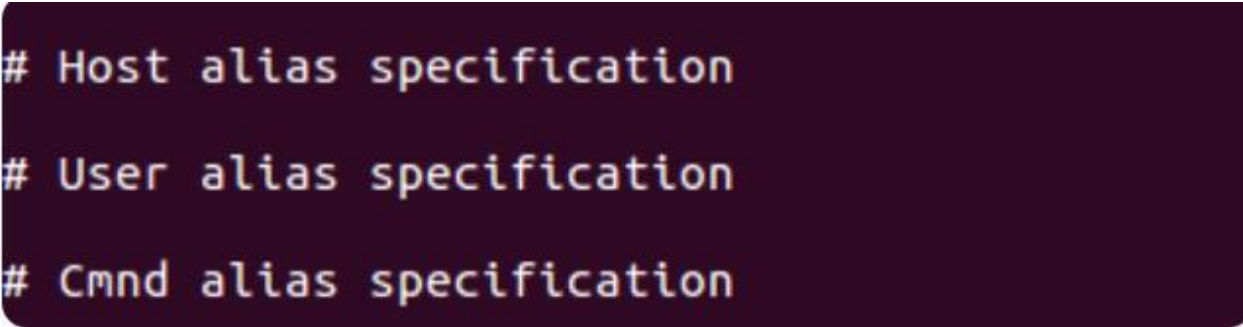
Если необходимо «накрыть» большое количество пользователей или команд, то можно ввести алиасы. Делается это следующим образом:

```
User_Alias SYSADMINS = john, tim, tom
```

То есть мы используем SYSADMINS сразу для пользователей john, tim, tom. Тогда написание правила будет выглядеть следующим образом:

```
SYSADMINS ALL = (root) /usr/bin/cat /etc/shadow
```

По умолчанию у нас уже есть место для трех алиасов:



```
# Host alias specification
# User alias specification
# Cmnd alias specification
```

Но ничего не мешает нам создавать свои. Имя алиаса должно начинаться с большой буквы, может содержать нижнее подчеркивание, буквы обоих регистров, цифры.

Кейс 5. Исключение

Применительно к пользователям, командам и группам, вы можете добавлять ! перед именами, таким образом вы можете обозначать «Разрешено всем, кроме X». Рассмотрим пример:

Создаем алиас пользователей, в котором есть ALL, и НЕ root:

```
User_Alias EXCEPT_ROOT = ALL, !root
```

А затем используем этот алиас в нужной нам команде:

```
skill ALL = (EXCEPT_ROOT) /usr/sbin/apache2
```

Редактировать файл `sudoers` лучше с помощью утилиты `visudo` — это считается безопасным способом в силу того, что так утилита автоматически проверяет написанное вами и не даст вам сохранить файл, если вы сделали синтаксическую ошибку.

Управление пользователями, их доступом и разрешениями — очень важное умение, которое используется повсеместно как системными администраторами, так и специалистами по информационной безопасности. Далее разберем еще один немаловажный инструмент, который облегчит вам жизнь (но с непривычки, конечно, сделает её немного сложнее).

Bash скрипты

Bash (*Bourne-again shell*) — это командный интерпретатор (или оболочка) и скриптовый язык программирования, разработанный проектом *GNU*. Используется в большинстве *Linux*-систем, но не является единственным (к примеру, ещё есть *zsh*). Для дистрибутива *Kali*, Bash является командной оболочкой по умолчанию.

Поведение *bash* зависит от того, каким образом он был запущен. Есть два варианта вызова *bash*: командная оболочка при логине пользователя в *TTY* (*login shell*), например, с помощью *SSH* (т.е. по факту вызван демоном *SSH*), и интерактивная оболочка (например, при вызове скрипта (*non-login shell*) или использования эмулятора терминала из графической оболочки (*interactive*)). В таблице представлены основные конфигурационные файлы:

Файл	Описание	Командная оболочка при логине (применимость)	Интерактивная командная оболочка (применимость)
<code>/etc/profile</code>	Задаёт элементы общесистемного окружения пользователя.	+	-
<code>~/.bash_profile</code>	Используется «персонально» после применения <code>/etc/profile</code> . Так же, как и <code>/etc/profile</code> задаёт свои общесистемные переменные окружения.	+	-
<code>~/.bash_logout</code>	Используется «персонально», задаёт действия после того, как пользователь покинет оболочку.	+	-
<code>/etc/bash.bash_logout</code>	То же самое, что и <code>bash_logout</code> выше, только применяется ко всем пользователям.	+	-
<code>/etc/bash.bashrc</code>	Применяется при логине пользователя.	-	+
<code>~/.bashrc</code>	«Персональное» применение при логине пользователя (например, можно удобно задать <i>alias</i>).	-	+

Что такое **переменные среды**? Это именованный объект, содержащий данные, которые будут использоваться в одном или нескольких приложениях. Грубо говоря, это переменная с именем и значением. Чтобы просмотреть текущие переменные среды, можно воспользоваться утилитой *printenv*.

```

root@kali:~# printenv
SHELL=/bin/bash
COLORTERM=truecolor
SUDO_GID=1000
LANGUAGE=
SUDO_COMMAND=/bin/bash
SUDO_USER=kali
PWD=/root
LOGNAME=root
XAUTHORITY=/home/kali/.Xauthority
LS_OPTIONS=--color=auto
HOME=/root
LANG=en_US.UTF-8
LS_COLORS=rs=0:di=01;34:ln=01;36:mh=00:pi=40;33:so=01;35:do=01;35:bd
:ex=01;32:*tar=01;31:*tgz=01;31:*arc=01;31:*arj=01;31:*taz=01;3
01;31:*zip=01;31:*z=01;31:*dz=01;31:*gz=01;31:*lrz=01;31:*lz=0
2=01;31:*tz=01;31:*deb=01;31:*rpm=01;31:*jar=01;31:*war=01;31:*
1:*rz=01;31:*cab=01;31:*wim=01;31:*swm=01;31:*dwm=01;31:*esd=0
*:pgm=01;35:*ppm=01;35:*tga=01;35:*xbm=01;35:*xpm=01;35:*tif=01
pg=01;35:*mpeg=01;35:*m2v=01;35:*mkv=01;35:*webm=01;35:*webp=01
=01;35:*asf=01;35:*rm=01;35:*rmvb=01;35:*flc=01;35:*avi=01;35:*
*.emf=01;35:*ogv=01;35:*ogx=01;35:*aac=00;36:*au=00;36:*flac=00
=00;36:*wav=00;36:*oga=00;36:*opus=00;36:*spx=00;36:*xspf=00;36
TERM=xterm-256color
USER=root
DISPLAY=:0.0
SHLVL=1
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
SUDO_UID=1000
MAIL=/var/mail/root
_=/usr/bin/printenv

```

Особенности работы

Работая с командной оболочкой *bash*, следует учитывать, что командный интерпретатор — это гораздо больше, чем выполнить команду и получить результат. *Bash* поддерживает некоторые регулярные выражения. Например, вам надо вывести на экран содержимое файла */etc/passwd*.

Самое первое, что придёт вам на ум (и будет верным) это ввести `cat /etc/passwd` (да, разумеется, вместо *cat* можно использовать абсолютно любой текстовый редактор, например *vi* или *nano*). Однако возможности *bash* предполагают выполнение этой же команды таким образом: `cat /e*c/p*wd`. То есть мы использовали так называемую мультикарту `*` при обращении к файлу */etc/passwd*. Так же будет действовать, например `cat /e?c/p???d` (запомните такие вызовы, они пригодятся вам при обходе *Application firewall*). Например, если надо отобразить содержимое файлов в директории */etc*, то лучше так же воспользоваться `*`: `cat /etc/*`, а чтобы было удобнее читать (например, постранично), то можно использовать *less*. Для этого используется оператор передачи вывода одной команды на ввод другой, т.е. `cat /etc/* | less`.

Вот основные операторы *bash* для составления последовательности команд:

- `&&` — оператор «и» последовательного выполнения (успешно выполнил команду 1, затем выполняет команду 2 и т.д.);
- `||` оператор «или» последовательного выполнения (неуспешно выполнил команду 1, затем выполнил команду 2);
- `|` — оператор *pipe*. Используется для передачи вывода команды 1 на вход команды 2 (использовал выше на скриншотах, выполняя команду типа `cat файл | grep что-то в файле`);
- `;` — оператор последовательного выполнения команд, вне зависимости от их результатов (успешного завершения);
- `&` — оператор выполнения команды в фоновом режиме

Принципы автоматизации

С помощью *bash* можно автоматизировать некоторые свои задачи с помощью скриптов. Для тех, кто хочет максимально глубокого погружения в скриптинг на *bash*, от всей души рекомендую прочесть книгу [Advanced Bash-Scripting guide](#) (наверняка есть и приличные локализованные издания, но я сторонник читать подобные вещи в оригинале).

В других языках программирования принято начинать свой путь с «*Hello, world!*», а в *bash* всё начинается с `#!/bin/bash`. Только в первой строке *bash* скриптов символ `#` не воспринимается как комментарий. Сначала всегда указывается путь к командному интерпретатору (как правило, он располагается по пути `/bin/bash`). А что спрашивается писать дальше? Без паники. В здешних скриптах используются:

- Команды (как обычно пишем их в терминале - `ls`, `pwd`, `cd` и т.д);
- Переменные (пользовательские или окружения, например, `$HOME`);
- Условия и математические операции;
- Циклы;
- Функции.

Практика написания скриптов

Давайте попробуем написать свой самый первый скрипт (как раз с приветствием этого прекрасного мира)

```
#!/bin/bash
echo hello world
```

Ок. А теперь давайте усложним задачу и подставим подпись того, кто поприветствовал мир:

```
#!/bin/bash
echo hello world from
whoami
```

Добавим переменную среды и пользовательскую:

```
#!/bin/bash
variable=$(date) #объявление пользовательской переменной
(да, это команда)
echo hello world from
whoami
echo my home at $HOME #тут добавлен комментарий, который не
будет воспроизведён скриптом, $HOME — переменная среды
echo сейчас $variable
```

Отлично. Далее давайте попробуем создать скрипт с простыми управляющими конструкциями типа *if, then, else*.

```
#!/bin/bash
if ping -c 2 8.8.8.8
then echo "интернет есть"
else echo "интернета нет"
fi
```

Вот результат работы такой простенькой проверки интернета (у некоторых может не работать с `sh`, поэтому рекомендую использовать `bash` имя скрипта):

```
root@kali:~# sh 1.sh
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.

--- 8.8.8.8 ping statistics ---
2 packets transmitted, 0 received, 100% packet loss, time 1014ms

интернета нет
root@kali:~# sh 1.sh
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=128 time=20.0 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=128 time=20.0 ms

--- 8.8.8.8 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1002ms
rtt min/avg/max/mdev = 19.961/19.998/20.036/0.037 ms

интернет есть
```

В скриптах можно сравнивать числовые и строчные значения. Сравнение чисел:

- `n -eq m` — возвращает истинное значение, если `n` равно `m`;
- `n -ge m` — возвращает истинное значение, если `n` больше или равно `m`;
- `n -gt m` — возвращает истинное значение, если `n` больше `m`;
- `n -le m` — возвращает истинное значение, если `n` меньше или равно `m`;

- `n -lt m` — возвращает истинное значение, если `n` меньше `m`;
- `n -ne m` — возвращает истинное значение, если `n` не равно `m`.

Сравнение строк:

- `string1 = string2` — проверяет строки на равенство, возвращает истину, если строки идентичны;
- `string1 != string2` — возвращает истину, если строки не идентичны;
- `string1 < string2` — возвращает истину, если `string1` меньше, чем `string2`;
- `string1 > string2` — возвращает истину, если `string1` больше, чем `string2`;
- `-n string1` — возвращает истину, если длина `string1` больше нуля;
- `-z string1` — возвращает истину, если длина `string1` равна нулю;

В сравнении строк есть один очень важный нюанс: необходимо экранировать оператор сравнения `<` и `>` с помощью `\`.

Примеры сравнения строк:

Пример 1.

```
#!/bin/bash
string1=$(whoami)
string2=root
if [ string1=string2 ] #так будет правильнее использовать
конструкцию сравнения пользователей
then echo you are root
else echo you are not root
fi
```

Пример 2.

```
#!/bin/bash
string1="verylongnamelongname"
string2="shortname"
if [ $string1 \< "$string2" ]
then echo "длина вашего имени короче переменной string1"
else echo "длина вашего имени длиннее переменной string1"
fi
```

Параметры

В примерах мы задавали все переменные в самом скрипте напрямую, либо результатом выполнения какой-либо команды. Однако при практическом использовании в `bash`-

скрипты можно (и нужно) передавать параметры. Давайте попробуем теперь сравнить имя пользователя с помощью передачи параметра:

```
#!/bin/bash
string1="root"
if [ $string1 = $1 ]
then echo "вы написали имя root правильно"
else echo "вы написали имя root неправильно"
fi
```

Обращаю ваше внимание, что переменная `$1` нигде не задается в явном виде, переменная `$0` — это имя скрипта. `$1` — первый параметр (`$2` — второй и т.д.), который передаётся в наш скрипт. Параметры же передаются через пробел.

[illegible]

Взаимодействие параметров (например, если вы задали несколько параметров) может осуществляться с помощью операторов `$*` (записывает все параметры как одно значение) и `$@` (записывает все параметры как отдельные значения).

Цикл for и while

В *bash* скриптах, в основном, используются 2 цикла — *for* и *while*. Цикл *for* используется для перебора значений.

Пример 1.

```
#!/bin/bash
for digit in 1 2 3 4 5 6 7 8
do
echo цифра $digit
done
```

```
(root@kali)-[~]  
# sh 1.sh  
цифра 1  
цифра 2  
цифра 3  
цифра 4  
цифра 5  
цифра 6  
цифра 7  
цифра 8
```

Пример 2.

Очень часто цикл *for* используется для сканирования файлов и директорий.

```
#!/bin/bash  
for file in "$@" #тут я воспользовался оператором $@ - т.е.  
все значения, которые будут переданы параметром, будут  
восприниматься отдельно, в качестве примера /etc/*  
do  
if [ -d "$file" ]  
then  
echo "$file - директория"  
elif [ -f "$file" ]  
then  
echo "$file -файл"  
fi  
done
```

```
(root@kali)-[~]
# sh 1.sh /etc/*
/etc/adduser.conf -файл
/etc/adjtime -файл
/etc/alsa - директория
/etc/alternatives - директория
/etc/apache2 - директория
/etc/apparmor - директория
/etc/apparmor.d - директория
/etc/apt - директория
/etc/avahi - директория
/etc/bash.bashrc -файл
/etc/bash_completion -файл
/etc/bash_completion.d - директория
/etc/bindresvport.blacklist -файл
/etc/binfmt.d - директория
/etc/bluetooth - директория
/etc/ca-certificates - директория
/etc/ca-certificates.conf -файл
/etc/calendar - директория
/etc/chatscripts - директория
/etc/cifs-utils - директория
/etc/cloud - директория
/etc/console-setup - директория
/etc/cron.d - директория
/etc/cron.daily - директория
/etc/cron.hourly - директория
/etc/cron.monthly - директория
/etc/crontab -файл
```

Цикл while

Цикл *while* позволяет задать проверку какого-либо условия и выполнять цикл до ошибки либо успешного завершения. В цикле могут быть использованы операторы *break* и *continue*.

Break — завершает текущий цикл и позволяет программе выполняться дальше (выход из цикла).

Continue — завершает текущую итерацию цикла и осуществляет переход к следующей.

```
#!/bin/bash
i=$1
while [ $i -lt 5 ]
do
echo "Number: $i"
((i++))
if [[ "$i" == '4' ]]; then
break
fi
done
echo 'я посчитал до 4'
```

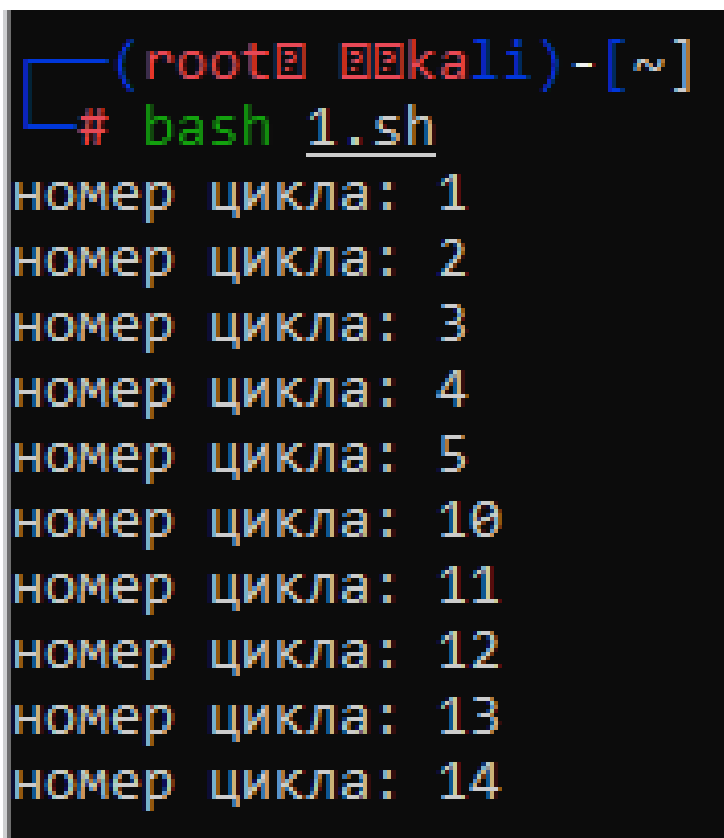
```
(root@kali)-[~]
# bash 1.sh 2
Number: 2
Number: 3
я посчитал до 4
```

```
(root@kali)-[~]
# bash 1.sh 1
Number: 1
Number: 2
Number: 3
я посчитал до 4
```

```
(root@kali)-[~]
# bash 1.sh 0
Number: 0
Number: 1
Number: 2
Number: 3
я посчитал до 4
```

Пример с *continue*. Пропускаем 4 цикла.

```
#!/bin/bash
for (( i = 1; i < 15; i++ )) #можно сделать бесконечным,
убрав второе условие
do
if [ $i -gt 5 ] && [ $i -lt 10 ]
then
continue
fi
echo "номер цикла: $i"
done
```



```
(root@kali) ~#
# bash 1.sh
номер цикла: 1
номер цикла: 2
номер цикла: 3
номер цикла: 4
номер цикла: 5
номер цикла: 10
номер цикла: 11
номер цикла: 12
номер цикла: 13
номер цикла: 14
```

Функции

Функция — это «именованный» блок кода, который можно повторно вызывать в скрипте. Важно объявить функцию до её вызова, иначе скрипт будет завершён с ошибкой. Обычно, функция задается очень просто:

```
function имяфункции {
код
}
```

Имя функции следует задавать уникальное. В противном случае функция может не отработать так, как планировалось (в случае коллизии имён функций, скорее всего, будет

перезапись функции, которая была выше по коду, той, которая оказалась ниже). Результатом выполнения той или иной функции является целое число до 255. Чтобы влиять на результат выполнения функции, используют команду `return`.

А как же функции возвращать что-то кроме чисел (например, текст)? Эта задача решается записью вывода функции в глобальную переменную. Кроме глобальной переменной есть локальная, подробнее о них:

- **Глобальные** — их использование не зависит от места объявления (внутри функции или в основном теле скрипта). Можно задать глобальную переменную в теле скрипта и использовать в функции, и, наоборот, можно задать глобальную переменную в функции и использовать далее (после вызова функции, разумеется) в теле скрипта;
- **Локальные** — используются только внутри функции и за её пределы не выходят (т.е. если в одной функции задали локальную переменную `loc_var`, то обратиться к её значению можно будет только в самой функции, вне функции или в других функциях она не будет `видна`). Локальные переменные объявляются с помощью `local имя_переменной`.

Функция может работать с аргументами, которые в неё передаются либо из тела скрипта, либо как пользовательский ввод (те же `$1, $2...`).

```
#!/bin/bash
function summ{ #объявляем функцию
echo $(( $1 + $2 )) # результатом работы функции будет
сложение двух чисел
}
if [ $# -eq 2 ] #считаем количество аргументов, переданных в
скрипт
then
global_variable=$(summ $1 $2) #присваиваем переменной
значение функции и передаем в неё аргументы
echo "Сложим 2 числа и получим $global_variable"
else
echo "Функции следует передать 2 числовых аргумента через
пробел"
fi
```

```
root@kali:~# bash 1.sh
Функции следует передать 2 числовых аргумента через пробел
root@kali:~# bash 1.sh 10 14
Сложим 2 числа и полчуим 24
```

В примере скрипта выше, мы можем вернуть значение функции гораздо больше, чем 255 из-за использования переменных.

```
root@kali:~# bash 1.sh 10000 200000
Сложим 2 числа и получим 210000
```

Итоги

По итогу этой главы мы с вами познакомились с архитектурой операционной системы *Linux*. Узнали, какие бывают дистрибутивы, научились управлять пакетами и писать скрипты в *bash*.

Потребность в понимании архитектуры операционной системы и написании скриптов — это база при решении задач по пентесту на CTF. Теперь вы можете достаточно уверенно (на уровне чуть выше джуна) ориентироваться в архитектуре *Linux*, выбирать файловые системы, управлять пакетами операционной системы и даже писать собственные скрипты.

В заключении главы хочется ещё раз напомнить тезис, который я вербально (а порой и весьма эмоционально) транслирую практически через все свои видео. «Грамотный специалист по информационной безопасности — это практически всегда в прошлом опытный системный администратор». И чтобы таковым стать навыки работы с операционной системой *Linux* необходимо совершенствовать всё время, не боясь огромного объёма технической документации и решения трудных задач.

Конец ознакомительного фрагмента. Полную версию книги можно купить по ссылке: <https://kurets.ru/linux-for-hackers>